

Leveraging Discrete CKKS to Bootstrap in High Precision

Hyeonmin Choe
CryptoLab Inc.
Seoul, Republic of Korea
hyeonmin.choe528@gmail.com

Damien Stehlé
CryptoLab Inc.
Lyon, France
damien.stehle@cryptolab.co.kr

Jaehyung Kim
Stanford University
Stanford, CA, USA
jaehk@stanford.edu

Elias Suvanto
University of Luxembourg
Esch-sur-Alzette, Luxembourg
elias.suvanto@uni.lu

Abstract

The CKKS fully homomorphic encryption (FHE) scheme enables computations on vectors of approximate complex numbers. A moderate precision of ≈ 20 bits often suffices but, in many applications, a higher precision is required for functionality and/or security. Indeed, to obtain IND-CPA-D security [Li-Micciancio; Eurocrypt'21], secure threshold-FHE [Asharov et al; Eurocrypt'12] and circuit privacy [Gentry; STOC'09], all known approaches require a precision that supports noise flooding. This may lead to a precision of ≈ 80 bits, or more. High-precision CKKS is hard to achieve, notably because of bootstrapping. The main difficulty is modulus consumption: every homomorphic multiplication consumes some, out of an overall modulus budget. Unfortunately, in high precision, most known bootstrapping algorithms consume so much modulus that one needs to increase the parameters to increase the budget. The state-of-the-art approach, Meta-BTS [Bae et al; CCS'22], performs moderate-precision bootstrapping several times to enable high-precision bootstrapping, with similar modulus consumption as the base bootstrapping it builds upon. It however damages latency.

We introduce a new approach for high-precision CKKS bootstrapping, whose cost is almost independent of the precision (as opposed to Meta-BTS) and whose modulus consumption increases significantly more slowly than with classical bootstrapping algorithms. Our design relies on the EvalRound bootstrapping [Kim et al; Asiacrypt'22], which we improve in the high-precision context by leveraging and improving recent techniques for handling discrete data with CKKS. We obtain for the first time a non-iterative 80-bit precise bootstrapping algorithm which can be run in ring degree $N = 2^{16}$, with 494 bits of remaining modulus for computations. In terms of throughput, and for 80-bit precision, our implementation shows an acceleration of 64% compared to Meta-BTS.

CCS Concepts

• Security and privacy → Cryptography.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CCS '25, Taipei, Taiwan

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1525-9/2025/10
<https://doi.org/10.1145/3719027.3765080>

Keywords

Homomorphic Encryption, CKKS, Bootstrapping, High Precision

ACM Reference Format:

Hyeonmin Choe, Jaehyung Kim, Damien Stehlé, and Elias Suvanto. 2025. Leveraging Discrete CKKS to Bootstrap in High Precision. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25)*, October 13–17, 2025, Taipei, Taiwan. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3719027.3765080>

1 Introduction

CKKS [21] is one of the most popular Fully Homomorphic Encryption (FHE) schemes, in particular when it comes to computations on real numbers. These real numbers are in fact computed upon with some precision, similarly to fixed-point arithmetic. This precision can be parameterized, with homomorphic computations getting more expensive with higher precision. In many secure delegations of computations, such as inference and training in privacy-preserving machine learning, moderate precision suffices. For instance, the main libraries providing implementations of CKKS [2, 25, 35] all provide a precision of ≈ 20 bits as a default option or one of the default options. This precision choice is in part guided by its compatibility with 64-bit integer computations when using RNS-CKKS [20]. In this work, we instead focus on higher precision computations.

In the context of FHE over real numbers, high precision may arise for specific computations that also require very high accuracy when performed in clear, for example, in the case of numerically unstable tasks. However, more commonly, the need of high precision in FHE computations stems from the use of stronger security properties than IND-CPA security or extended functionality. Let us mention three such security properties. First, circuit privacy [29] requires that the decryptor should not be able to grasp information about the evaluated circuit beyond what it can get by making evaluation requests; this can be a desirable property to protect model weights in secure inference. Second, IND-CPA-D security [38] requires that the scheme retains IND-CPA security even if the adversary can see the plaintexts obtained by decrypting legitimately formed ciphertexts; this is particularly relevant if the client publishes the result of the computation it securely delegated, such as models obtained from sensitive data but that are meant to be publicly used. Finally, in the case of Threshold-FHE [5, 9, 43] and related notions (see, e.g. [15, 41, 44]), the decryption key is secret-shared among users and one requires that the scheme retains IND-CPA security even if the adversary corrupted some fraction of the secret key

share-holders. In all these cases, the only known general-purpose solutions for CKKS rely on exponential noise flooding: as the decryption noise contains sensitive information, one hides it by adding a statistically fresh noise to it, which is exponentially larger as a function of the security parameter λ . Why does it lead to using high-precision computations in CKKS? In CKKS, noise and plaintext are tied together: the real-valued plaintext is known up to some accuracy, and the inaccuracy is the noise. Therefore, to hide the noise with exponential noise flooding while retaining useful information in the most significant plaintext bits, one aims at a relative error (before flooding) that is exponentially smaller than the plaintext values, or, put differently, a numerical precision that grows linearly in λ . If the data consists of very small integers, then it may be made more accurate thanks to cleaning [26]. Otherwise, one is led to using high precision throughout the whole computation, from the very start. Concretely, depending on the bit-security that one aims to obtain with flooding, one can typically request from 40 to 64 extra bits of precision, on top of the desired output precision [39].

Handling such large precisions in CKKS has proved challenging. The main difficulty comes from the modulus consumption of bootstrapping. In CKKS, a ciphertext (resp. evaluation key) is an element (resp. a family of elements) of $\mathcal{R}_q^2$ where $\mathcal{R}_q = \mathbb{Z}_q[x]/(x^N + 1)$ for an integer $q > 1$ and a power-of-two integer $N > 1$. The ciphertext modulus q evolves during homomorphic evaluation: every homomorphic multiplication decreases q by a factor $\approx 2^t \cdot C$, where t is the plaintext precision and C depends on other scheme parameters. To ensure that the Ring-LWE instances [42, 46] provide sufficient security, for any value of N , the modulus q cannot exceed some amount. For example, for $N = 2^{16}$, the bit-size of q should be at most ≈ 1600 , the exact value depending on the specific Ring-LWE parametrization. To enable arbitrarily long computations, CKKS bootstrapping [19] takes as input a ciphertext with a small modulus and maps it to a ciphertext that decrypts to the same plaintext, up to some small error, but defined over a much larger modulus. Despite numerous works on CKKS bootstrapping (see, among others, [10, 11, 14, 30, 37]), state-of-the-art bootstrapping still requires of the order of 20 multiplicative levels (including auxiliary modulus used in key switching) and consumes of the order of 1150 bits for a plaintext precision $t \approx 20$ bits (see, e.g., [10]). Increasing t by 40 to 64 bits would increase bootstrapping modulus consumption by 800 to 1280 bits. Proceeding in this way is incompatible with taking $N = 2^{16}$, as the required modulus is too large for Ring-LWE security; this can be accommodated by increasing the Ring-LWE degree, but with a significant performance impact.

An alternative to increasing the plaintext precision is to simulate high-precision plaintexts using smaller-precision plaintexts, like in arbitrary-precision real-number software. This can be realized homomorphically using the tuple-CKKS method introduced in [16]: concretely, increasing the precision from 20 to $20 + 40 = 60$ or $20 + 60 = 80$ bits can be achieved by representing plaintexts with 3 or 4 ciphertexts with 20-bit precision each. This allows lowering the modulus consumption, perhaps making it possible to bootstrap high plaintext precisions with $N = 2^{16}$, at the expense of manipulating more ciphertexts in parallel. Finally, the high-precision bootstrapping algorithm from [6], available in [2, 35], takes a different route. It consists of running small-precision bootstrapping

several times sequentially, to progressively increase the precision of the result. Combined with tuple-CKKS or composite rescaling [1] in RNS-CKKS, this provides a full-fledged solution for high-precision CKKS. For 60 to 80 bits of plaintext precision, it, however, requires at least 3 to 4 sequential 20-bit bootstraps.

In all the above approaches, increasing the plaintext precision from a constant to t bits leads to an increase of the bootstrapping cost by (at least) a factor $O(t)$. Can we bootstrap in high plaintext precision with a lower cost penalty?

Contributions. We introduce a novel bootstrapping algorithm that significantly improves the performance of CKKS bootstrapping in the case of high-precision plaintexts. Compared to the previous approaches, its cost and modulus penalties, compared to small precision bootstrapping, are almost only *additive* rather than multiplicative. In more detail, CKKS bootstrapping algorithms all consist of homomorphic Discrete Fourier Transformations called coeffs-to-slots (CtS) and slots-to-coeffs (StC), and of a non-linear component that can come in several flavors. In our case, out of these three main bootstrapping components, only the modulus consumption of the cheapest one, StC, grows linearly in t where t is the plaintext precision. The cost and modulus consumption of CtS do not increase with t , and those of the non-linear bootstrapping components grow slowly with t (modulus consumption increases by an additive term $O(t)$ as opposed to a multiplicative factor).

Our algorithm builds upon the EvalRound/EvalRound+ bootstrapping algorithms [33, 47]. These differ from other bootstrapping algorithms in their non-linear component: instead of evaluating a modular reduction $I + x \mapsto x$ where I is a bounded integer and x a real number with $|x| \ll 1$ (EvalMod), they evaluate the rounding function $I + x \mapsto I$ (EvalRound). This reduces the modulus consumption of CtS. These works however implement EvalRound as $\text{id} - \text{EvalMod}$. Our main technical ingredient is a new EvalRound algorithm whose cost and modulus consumption depend only additively in the plaintext precision. For this purpose, we borrow and improve techniques introduced for handling discrete data with CKKS [4, 7, 8, 12, 24, 26]: indeed, the plaintext $I + x$ is a noisy representation of a small integer I . We break I into pieces, to reduce the task to increasing the precision of bits (elements of $\{0, 1\}$) or trits (elements of $\{-1, 0, 1\}$): when the plaintext precision increases, this requires high precision only in the final steps.

We further investigate the bit/trit cleaning step, as it is a significant contributor to the total modulus consumption of our bootstrapping algorithm. We implement it by repeatedly evaluating a precision-doubling polynomial. The polynomials we choose require multiplicative depth two. As a result, if the plaintext precision increases from t to $\approx 2t$, then a black-box implementation — from homomorphic addition, multiplication and conjugation — consumes $4t$ bits of modulus (i.e., $2t$ bits per multiplicative level). Instead, we open the black-box and use the sub-components of homomorphic multiplication to design an evaluation of these polynomials that consumes only t bits of modulus. In practice, this improvement saves hundreds of bits in modulus consumption.

Implementation and Experiments. We implemented our algorithm using a version of the HEaAN library [25] that relies on the recent grafting technique from [17]. This technique is particularly useful as our algorithm relies on many different (small and large)

precisions. We note that this is an experimental code, and its efficiency can most likely be improved further. It should however provide a meaningful comparison to the state-of-the-art approach, Meta-BTS, as we estimate its time based on our implementation.

The concrete modulus consumption of our algorithm for ring degree $N = 2^{16}$ is highlighted in Table 1, for plaintext precisions ranging from 20 to 80 bits. In particular, it is the first time that a high-precision non-iterative bootstrap is reported in ring degree $N = 2^{16}$. The timings of the first four rows are in a single-thread CPU (Apple M4 Pro, 32GiB RAM), using the same code-base; the data of the fifth row is extrapolated from the fourth; we do not give timings for the last row as [47] uses a different (less efficient) code-base. The first three parameter sets correspond to our algorithm, FGb* is a modified parameter set of [25] that provides a similar failure probability $\approx 2^{-128}$ as our parameter sets.

Table 1: Modulus consumptions and execution times for various CKKS bootstrapping algorithms. The first three rows are for our bootstrapping, and the fourth row is for the baseline CKKS bootstrapping. The fifth and sixth rows are for using Meta-BTS [6], and the last row is for EvalRound+ [47]. “Prec.” refers to the precision of the bootstrap, while “Available modulus” refers to the modulus that is left after bootstrapping.

Parameter set	Cleaning iterations	Prec.	Available modulus	Time (s)	Modulus / Time
Direct2Cln19	2	19	742	14.3	51.8
Direct2Cln38	2	38	640	17.9	35.8
Direct3Cln81	3	81	494	20.9	23.7
FGb*	-	21	550	8.5	64.6
FGb* w/ Meta-BTS $\times 2$	-	≈ 40	530	17.0	31.1
FGb* w/ Meta-BTS $\times 4$	-	≈ 80	490	34.0	14.4
EvalRound+	-	16	546	$-^1$	-

1.1 Technical Overview

Consider a ciphertext $(a, b) \in \mathcal{R}_{q_0}^2$ such that $a \cdot s + b \approx \Delta \cdot m \bmod q_0$, where $s \in \mathcal{R} = \mathbb{Z}[x]/(x^N + 1)$ is the secret key, $\Delta \ll q_0$ is a scaling factor growing as $C \cdot 2^t$ for plaintext precision t and with C depending on other scheme parameters, and $m \in [-1, 1]^N$ is the plaintext. CKKS bootstrapping regains modulus by viewing (a, b) as a no-modulus ciphertext: there exists $I \in \mathcal{R}$ such that $a \cdot s + b = q_0 \cdot I + \Delta \cdot m$ over $\mathcal{R}$ and hence over $\mathcal{R}_Q$ for any $Q > 1$. This re-interpretation of (a, b) is called ModRaise. The rest of CKKS bootstrapping aims at removing $q_0 \cdot I$ from $q_0 \cdot I + \Delta \cdot m$ while approximately keeping m , to obtain a high-modulus ciphertext of m . Classical CKKS bootstrapping achieves it by viewing $q_0 \cdot I + \Delta \cdot m$ as a higher-precision plaintext and performing a homomorphic reduction modulo q_0 implemented with a polynomial approximation.

¹[47] reported 1,070 seconds using a different code-base, which is not directly comparable.

Extending the above to high-precision plaintexts leads to higher precision computations all throughout bootstrapping, as the bootstrapping plaintexts should be sufficiently precise to keep m accurate in $q_0 \cdot I + \Delta \cdot m$. Instead, we start from the observation from [31, 33, 47] that it suffices to bootstrap I to bootstrap m . Indeed, given $(a, b) \in \mathcal{R}^2$ and $(a', b') \in \mathcal{R}_Q^2$ such that²

$$a \cdot s + b = q_0 \cdot I + \Delta \cdot m \quad \text{and} \quad a' \cdot s + b' \approx q_0 \cdot I \bmod Q,$$

then $(a - a', b - b') \in \mathcal{R}_Q^2$ satisfies

$$(a - a') \cdot s + (b - b') \approx \Delta \cdot m \bmod Q.$$

Note that the plaintext precision of (a', b') must be sufficiently high so that m is accurately stored in $(a - a', b - b')$. Bootstrapping I is considered in [31] to reduce a CKKS bootstrap to N bootstraps of DM/CGGI ciphertexts [23, 27]. Because of the magnitude of N , this is mostly of theoretical interest. In [33, 47], bootstrapping I is used to reduce the plaintext precision of the first bootstrapping component that follows ModRaise, namely CtS. To see this, recall that the native CKKS operations operate on slots (i.e., in the Fourier domain), and that the purpose of CtS is to bring $q_0 \cdot I + \Delta \cdot m$ (resp. $q_0 \cdot I$) from the coefficients to the slots, so that subsequent slots operations can remove I (resp. increase the accuracy of I). After CtS, the bootstrapping algorithms from [33, 47] however use high plaintext precision all-throughout their EvalRound procedure to obtain the ciphertext (a', b') above.

Let us now see how EvalRound can be implemented. Recall that the goal is to start from a ciphertext encrypting I with low accuracy (in the slots) and obtain a ciphertext (a', b') also decrypting to I but with high accuracy. In [33, 47], this is achieved using $\text{id} - \text{EvalMod}$, where EvalMod is the modular reduction function from conventional CKKS bootstrapping [19], which uses for all its steps a plaintext precision that is no smaller than the target precision. This approach ignores that I is a small integer. For example, using the sparse secret encapsulation technique from [11], for $N = 2^{16}$, we have that $\|I\|_\infty \leq 15$ with probability extremely close to 1, under the assumption that the a -part of the ciphertext to be bootstrapped is uniformly distributed. In fact, increasing the accuracy of an integer I has been independently considered as a specific task in [26], which focused on manipulating discrete data with CKKS: increasing the precision of I is viewed as a cleaning process, allowing further discrete data computations.

We also view EvalRound as an integer cleaning process, but rely on a different algorithm from [26]. In that work, the authors extract the most significant bit using a sign function evaluation [22], then clean that bit, subtract it from the integer and continue with the next most significant bit. The use of [22] and the sequential aspect incur huge multiplicative depth, which is unsuitable for bootstrapping. We extract and clean the bits of I in parallel. The bit extraction can be performed directly with polynomial interpolation. We also consider an alternative approach, inspired by the use of complex roots of unity in [4, 7, 12, 24]: we first evaluate a complex exponential $I \in \mathbb{Z} \mapsto \exp(2i\pi I / (2K + 1)) \in \mathbb{C}$ where K is an upper bound on $|I|$, and then extract the bits of I from $\exp(2i\pi I / (2K + 1))$ using the algorithm from [7]. Once we have the bits, stored in a few ciphertexts, we can clean them as

²The astute reader will notice that this formulation is better suited to CtS-first bootstrapping. We focus on that variant in the introduction, for the sake of simplicity.

in [26], i.e., using the map $h_1 : x \mapsto 3x^2 - 2x^3$ from [22]. The cleaning is performed iteratively, until the target precision is obtained. The main advantage of our integer cleaning process is that most steps require only very small plaintext precision: indeed, bit extraction only aims to obtain bits. A cleaning iteration consumes two multiplicative levels and doubles the precision: overall, the modulus consumption of cleaning is essentially four times that of the output precision (two multiplicative levels in maximal precision, preceded by two multiplicative levels in half that precision, etc). Finally, we can recombine the cleaned bits

Let us now summarize our bootstrapping process for $(a, b) \in \mathcal{R}^2$ such that $a \cdot s + b = q_0 \cdot I + \Delta \cdot m$.

- (1) **Coefficients-to-slots.** As the small integer polynomial I is in the coefficients, we use CtS to put it in slots. The plaintext precision depends only on the maximal bit-size of I , not on the precision of the message m .
- (2) **Bit extraction.** Using the fact that $\|I\|_\infty$ is small, we bit-decompose it (coordinate-wise) and put its bits across several ciphertexts. The plaintext precision depends only on the maximal bit-size of I .
- (3) **Iterative cleaning.** We repeatedly apply the h_1 map to clean each bit, to reach a precision that is a little above that of m . The total modulus consumption is approximately 4 times the precision of m .
- (4) **Recombination.** We recombine the bits into an integer that represents I . Assuming the bits are properly scaled, this is an addition.
- (5) **Slots-to-coefficients.** We put I back into coefficients using StC, resulting in an encryption of $q_0 \cdot I$ in coefficients. The plaintext precision is a little more than that of m .
- (6) **Subtraction.** We subtract $q_0 \cdot I$ from the original $q_0 \cdot I + \Delta \cdot m$, getting $\Delta \cdot m$ as desired.

Optimizations. We now discuss several improvements of the bootstrapping procedure above.

First, we note that it is compatible with the skip connection technique from [47], which allows to obtain an StC-first bootstrapping variant. In StC-first bootstrapping, StC is run before ModRaise, at the lowest ciphertext modulus possible, rather than at the end of bootstrapping with a higher modulus. This amounts to swapping StC and non-bootstrapping computations. In most applications, non-bootstrapping computations are lighter than StC and it is beneficial to lower the modulus at which StC is performed. However, the skip connection technique involves an extra CtS branch in parallel of the low precision CtS, extraction and cleaning: it has to be performed in high precision, but its cost remains limited overall as a large amount of ciphertext modulus is available to accelerate it.

Second, we focus on iterative cleaning, as this is a significant contributor to run-time and modulus consumption, in the case of high-precision plaintexts. First, we describe a way to clean using trits rather than bits. For this purpose, we use the cleaning function from [8], which has degree 3 like the h_1 map. This reduces the number of digits and hence of ciphertexts to be cleaned in parallel. Second, concerning modulus consumption, as each cleaning iteration squares the scaling factor Δ , a direct approach consists in increasing Δ to the target scaling factor Δ^2 before applying $h_1(x) = 3x^2 - 2x^3$ (or the trit-cleaning map). This results in a

modulus consumption of Δ^4 , as there are two multiplicative levels. This direct approach wastes modulus because it views homomorphic multiplication as a black box. Instead, we use subcomponents of homomorphic multiplication to lower the modulus consumption. Homomorphic multiplication consists of tensoring, relinearization, and rescaling. We view the underlying message at the beginning of cleaning as error-free, and observe that tensoring is error-free. Given a ciphertext ct encrypting a vector $\vec{z} \in \mathbb{C}^{N/2}$ with a scaling factor Δ , we may compute $ct' = \text{Relin}(ct \otimes ct)$ which decrypts to $\vec{z} \odot \vec{z}$ with scaling factor Δ^2 (here the notation $\odot$ refers to component-wise product). Note that achieving this already consumed Δ^2 of modulus in the direct method, while our approach does not consume any modulus so far. Next, we compute the product between ct and ct' whose tensor is also error-free. We save an extra factor Δ of modulus here. Finally, we rescale to lower the scaling factor from Δ^3 to Δ . Overall, this results in a massive reduction of modulus consumption from Δ^4 to Δ .

Figure 1 gives an overview of our implementation design for bootstrapping with 81 bits of precision, in terms of modulus consumption of the different components.

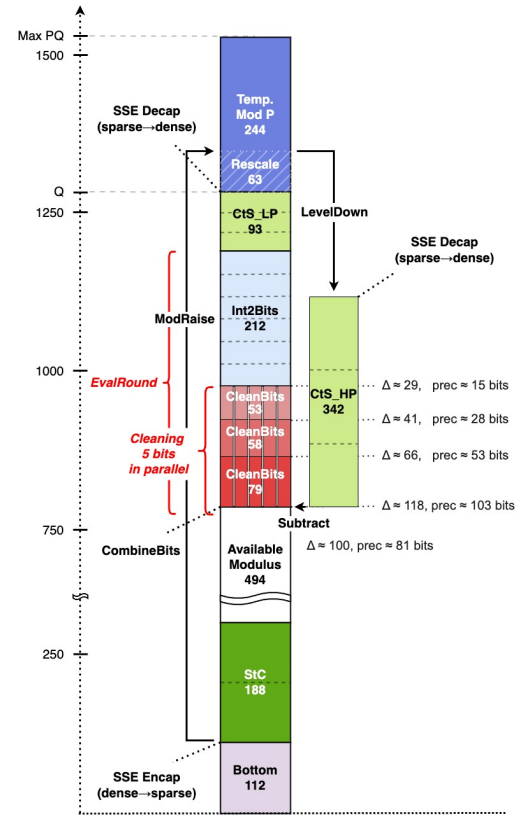


Figure 1: Overview of our bootstrapping implementation for 81-bit precision, parameter set Direct3Cln81. Modulus consumptions are shown for each sub-procedure, in bits.

Applications. As explained earlier in this introduction, using high-precision for general computations with CKKS is necessary for the only approach known to achieve circuit privacy, IND-CPA-D

security and secure threshold decryption: indeed, all known approaches require exponential noise flooding (as a function of the security parameter λ). It was even shown in [39] that if one adds a Gaussian noise, then its standard deviation must grow exponentially with λ (this was later extended to circuit privacy in [34]). The same proof technique directly applies to IND-CPA-D-secure CKKS: once exponential noise flooding is applied after the homomorphic computation with negligible failure probability, IND-CPA-D security follows. Since the flooding error is added at the end of the computation, only a few meaningful bits of the message remain, which in turn forces the entire homomorphic computation to be carried out at high precision—even for applications that would otherwise use a small scaling factor.

In line with this, we set all our implementation parameters so that the failure probability is $\varepsilon \leq 2^{-128}$, using the formula from [10, Section 6], as a larger ε may be exploited by the adversary [18]. The concrete impact of higher-precision bootstrapping is reflected in Table 1. However, the bootstrapping time alone is insufficient to quantify the full run-time cost for IND-CPA-D-secure CKKS, as non-bootstrapping computations must also be performed at higher precision to maintain correctness. A practical tool for this setting is tuple-CKKS[16], which enables linear scaling of computation time with respect to the target precision. As a rule of thumb, tuple-CKKS therefore provides a natural mechanism to extend high-precision requirements beyond bootstrapping.

Another application of high-precision CKKS bootstrapping was recently put forward in [32]: bootstrapping BFV [13, 28] ciphertexts. This approach requires exceptionally high plaintext precision (hundreds of bits), which was handled using META-BTS [6] with many iterations. If p refers to the BFV modulus and D the BFV multiplicative depth available after bootstrapping, then the CKKS plaintext precision is $\approx D \log p$. Our algorithm can be used as a drop-in replacement of META-BTS inside the BFV bootstrapping algorithm from [32].

Other related work. In a recent work [40], Li et al. proposed to replace the homomorphic evaluation of modular reduction used in conventional CKKS by homomorphic evaluation of the comparison function. The algorithm belongs to the “bootstrapping I family” [31, 47]. Given a ciphertext $(a, b) \in \mathcal{R}^2$ such that $a \cdot s + b = q_0 \cdot I + \Delta \cdot m$, and assuming K is a known bound on $\|I\|_\infty$, their strategy consists in homomorphically comparing any given coefficient of I to $-K, -K+1, \dots, K-1, K$ (using [22]). These comparisons output bits (namely, 0 if smaller than the value, 1 else), so that the shifted sum of the bits is equal to I . Our design strategy applies to this bootstrapping algorithm. Indeed, up to the comparison, everything can be performed with small plaintext precision, and then the bits returned by the comparisons can be made more accurate by iterative cleaning, before being summed. However, we note that this involves $2K+1$ parallel tracks for comparison and cleaning, as opposed to $O(\log K)$ in our case.

2 Preliminaries

Let $N > 1$ be a power-of-two integer and $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$. Given an integer $q > 1$, we define $\mathcal{R}_q = \mathcal{R}/q\mathcal{R}$. All our logarithms are in base 2. We let i be a square root of -1 and, for $x \in \mathbb{C}$, we let $\bar{x}$ denote the complex conjugate of x .

2.1 CKKS Basics

The CKKS fully homomorphic encryption scheme [21] relies on the Ring Learning With Errors (RLWE) problem [42, 46] for its security, and has a specific encoding structure (from cleartexts into ring elements) that enables SIMD computation over complex numbers. The discrete Fourier transform (DFT), from $\mathbb{R}[X]/(X^N + 1)$ to $\mathbb{C}^{N/2}$, is defined as

$$\text{DFT}(p(X)) = (p(\zeta^{5^j}))_{0 \leq j < N/2},$$

where $\zeta \in \mathbb{C}$ is a primitive $(2N)$ -th root of unity. The encoding map $\text{Ecd} : \mathbb{C}^{N/2} \rightarrow \mathcal{R}$ is defined as

$$\text{Ecd}_\Delta(\vec{z}) = \lfloor \Delta \cdot i\text{DFT}(\vec{z}) \rfloor,$$

where $i\text{DFT}$ is the inverse of DFT and $\Delta > 0$ is a scaling factor. As the DFT is a ring homomorphism, respectively mapping addition and multiplication over $\mathbb{R}[X]/(X^N + 1)$ to element-wise addition and multiplication over $\mathbb{C}^{N/2}$, the encoding map allows to rely on arithmetic over $\mathcal{R}$ to enable approximate SIMD additions and multiplications over the complex plane. The scaling factor Δ parametrizes the precision of those computations. In the context of CKKS, the coordinates of vectors in $\mathbb{C}^{N/2}$ are called slots, as opposed to the coefficients of elements of $\mathcal{R}$.

A coefficients-encoded CKKS ciphertext of $m \in \mathcal{R}$ is a pair of ring elements $(a, b) \in \mathcal{R}_Q^2$ such that $a \cdot s + b \approx \Delta \cdot m \bmod Q$, where s is the secret key, Q is the current modulus and Δ is the current scaling factor. The three parameters s , Q and Δ may vary depending on the homomorphic computation context. Similarly, a slots-encoded CKKS ciphertext of $\vec{z} \in \mathbb{C}^{N/2}$ is a pair $(a, b) \in \mathcal{R}_Q^2$ such that $a \cdot s + b \approx \Delta \cdot i\text{DFT}(\vec{z}) \bmod Q$.

Native operations. The elementary homomorphic operations enabled by CKKS, defined over $\mathbb{C}^{N/2}$, include slot-wise addition, multiplication and (complex) conjugation, as well as cyclic rotations of coordinates by an arbitrary amount. Addition, conjugation, and rotations map ciphertexts modulo Q to ciphertexts modulo Q : they preserve the ciphertext modulus. Oppositely, multiplication maps ciphertexts modulo Q to ciphertexts modulo $Q' \approx Q/\Delta \leq Q$: multiplication decreases modulus. Dividing the scaling factor and the ciphertext modulus of a ciphertext $(a, b) \in \mathcal{R}_Q^2$ by $\approx \Delta$ is called rescaling and denoted $\text{RS}_\Delta((a, b)) \in \mathcal{R}_{Q'}^2$.

Bootstrapping. For a fixed ring degree N , the ciphertext modulus Q is bounded from above because, otherwise, the underlying RLWE instances become insecure. Over the course of homomorphic computations, the ciphertext modulus may decrease, until reaching a bottom modulus $q_0 > 1$. The purpose of CKKS bootstrapping [19] is to regain modulus. Conventional CKKS bootstrapping includes four components, namely ModRaise, CtS, EvalMod and StC.

- (1) **ModRaise:** The modulus raising step is computationally vacuous. It regards a ciphertext (decrypting to $m \in \mathcal{R}$, i.e., with coefficients-encoding) at the bottom modulus q_0 as a ciphertext without modulus. Assuming that the input ciphertext is properly normalized (i.e., so that its coefficients belong to $[-q_0/2, q_0/2)$), the result of modulus raising is a ciphertext that decrypts to $m + q_0 \cdot I$ for some $I \in \mathcal{R}$ whose coefficients have small absolute values.

- (2) CtS and StC: The coefficients-to-slots and slots-to-coefficients steps homomorphically evaluate iDFT and DFT, respectively. This allows to convert a coefficients-encoded ciphertext into a slots-encoded ciphertext (resp. vice versa). These steps are used because the output of ModRaise is better defined over coefficients, while the last bootstrapping component, EvalMod, involves SIMD computations over slots.
- (3) EvalMod: This fourth component homomorphically evaluates a modular reduction function, to remove the term I in $m + q_0 \cdot I$. A typical approach is to use a polynomial approximation of the modular reduction function, over a range that includes all possible values of the coefficients of I .

The components can be combined in two ways. CtS-first bootstrapping starts at the bottom modulus q_0 , and can be described as $\text{StC} \circ \text{EvalMod} \circ \text{CtS} \circ \text{ModRaise}$. StC-first bootstrapping starts at a slightly larger modulus to enable StC before reaching the bottom modulus q_0 ; it can be described as $\text{EvalMod} \circ \text{CtS} \circ \text{ModRaise} \circ \text{StC}$. They differ in that CtS-first bootstrapping allows non-bootstrap computations at lower moduli than CtS-first bootstrapping, at the expense of performing StC at a higher ciphertext modulus: one may then be preferable over the other one depending on the relative costs of StC and the non-bootstrap computations.

Going further into details, we note that a ciphertext has $N/2$ complex-valued slots and N real-valued coefficients. As EvalMod requires real numbers in its slots, but I has N coefficients, EvalMod is preceded by a homomorphic extraction of real and imaginary parts (using complex conjugation), run twice in parallel, and followed by a recombination of the real and imaginary parts.

2.2 Ciphertext Arithmetic

CKKS ciphertext operations involve operations in $\mathcal{R}_Q$ for often large moduli Q (e.g., of 1500 bits) and large ring degrees (e.g., $N = 2^{16}$). To obtain reasonable performance, one is led to set Q to be a product $Q = q_1 \cdots q_k$ where the q_j 's are prime and congruent to 1 modulo $2N$. This modulus choice enables a *double-CRT decomposition*: first, elements in $\mathcal{R}_Q$ are represented as tuples in $\mathcal{R}_{q_1} \times \cdots \times \mathcal{R}_{q_k}$, via the Chinese Remainder Theorem; second, for each $j \leq k$, elements of $\mathcal{R}_{q_j}$ are handled as vectors in $\mathbb{Z}_{q_j}^N$ via the Number Theoretic Transform (NTT), which can be viewed as another application of the Chinese Remainder Theorem.

In [20], which was subsequently adopted in most libraries implementing CKKS, the q_j 's coincide with the rescaling involved in homomorphic multiplication. More concretely, as we saw earlier, homomorphic multiplication involves a modulus change from Q to a modulus Q' satisfying $Q' \approx Q/\Delta$: one can then choose $q_j \approx \Delta$ for all j and divide Q by one of its factors q_j to obtain Q' . This approach simplifies ciphertext arithmetic and creates significant parallelizability. On the downside, it ties the factorization of ciphertext moduli to the plaintext precision. If different plaintext precisions are required in a computation, then one can choose q_j 's of different magnitudes corresponding to these precisions, and consistently “peel-off” the ciphertext modulus Q during homomorphic computations. This approach has at least three significant drawbacks: it becomes cumbersome when multiple precisions are required; very small plaintext precisions may not be achievable in this way, as there are not many small q_j 's that are prime and congruent to 1

modulo $2N$; efficiency is sub-optimal when the plaintext precision is not compatible with q_i 's whose bit-lengths are just below 64 bits (or multiple thereof if using composite scaling [1]).

Our bootstrapping algorithm relies on multiple precisions, making the ciphertext arithmetic from [20] cumbersome and unnecessarily costly. Instead, we rely on the ciphertext arithmetic from [17] rather than the one from [20]. The work introduces a new technique called grafting, which decouples the arithmetic shape of the ciphertext moduli from the plaintext precision. It chooses most q_j 's to have just below 64 bits, thereby decreasing the number of factors that compose the ciphertext modulus Q . As a result, the FHE implementer can arbitrarily choose plaintext precisions (or, equivalently, scaling factors Δ), and they will be handled without loss of efficiency for ciphertext arithmetic. We note that a similar but somewhat less flexible approach was proposed in [45].

2.3 Discrete CKKS

In [26], Drucker *et al* proposed to rely on the inclusion $\{0, 1\}^{N/2} \subset \mathbb{R}^{N/2}$ to exploit the CKKS fully homomorphic encryption scheme for performing homomorphic binary computations. As all binary gates can be implemented as bivariate polynomials over $\mathbb{R}$, SIMD binary gates can be implemented by relying on the native CKKS addition and multiplication. As CKKS computations are intrinsically noisy, the slots may not be so close to 0 or 1 after the evaluation of several binary gates. In order to keep the deviation from the distinguished points 0 and 1 bounded, Drucker *et al* introduced a framework called “cleaning”, which reduces such errors by evaluating well-designed polynomials. For instance, consider $h_1(x) = 3x^2 - 2x^3$ (which had previously been considered in [22] in the context of homomorphic comparisons): as $h_1(0) = 0$, $h_1(1) = 1$ and $h_1'(0) = h_1'(1) = 0$, it preserves the underlying binary plaintext while essentially squaring the error. Elementary calculus provides the following result.

LEMMA 2.1. *For all $\beta \in \{0, 1\}$ and ε with $|\varepsilon| \leq 1$, we have:*

$$|h_1(\beta + \varepsilon) - \beta| \leq 5|\varepsilon|^2.$$

We discuss an extension to trits in Appendix A.

3 Proposed Bootstrapping

The conventional CKKS bootstrapping can be reformulated as a combination of embedding (i.e., ModRaise) and modular reduction (i.e., EvalMod). ModRaise raises the ciphertext modulus while adding a small multiple of q_0 denoted as $q_0 \cdot I$, whereas EvalMod homomorphically evaluates the modular reduction to remove it. The EvalRound [33] algorithm takes a slightly different point of view. Instead of removing the $q_0 \cdot I$ term directly, it reduces the associated noise and then subtracts the resulting ciphertext from the original one. At a high level, focusing on I obviates the need to maintain the plaintext message precision throughout bootstrapping. For instance, the main gain from [33] was achieved by lowering the precision of CtS. We go a step further by redesigning EvalRound (instantiated as $\text{id} - \text{EvalMod}$ in [33]).

3.1 Overview

We first recall the bootstrapping algorithm in [33] (see Algorithm 1). EvalRound takes as input a low-precision ciphertext that contains

an inaccurate integer vector $\vec{I} + \vec{e}$ in its slots, with $\|\vec{I}\|_\infty \leq K$ for some integer K and $\|\vec{e}\|_\infty \leq \varepsilon$ for some $\varepsilon \in (0, 1/8)$, and outputs a high-precision ciphertext that contains the very accurate integer vector $\vec{I} + \vec{e}'$ in its slots, i.e. $\|\vec{e}'\|_\infty \leq \varepsilon'$ for some $\varepsilon' \ll \varepsilon$. The bound ε' is set so that subtraction from a ciphertext decrypting to $q_0 \cdot \vec{I} + \Delta \cdot \vec{m}$ provides a ciphertext for $\vec{m}$ with the target plaintext precision. Recall (see the end of Section 2.1) that at this stage, bootstrapping operates on $N/2$ slots, but that there are N coefficients in the ring elements I created by ModRaise. This is handled by extracting the real and imaginary parts before running homomorphic rounding twice in parallel and then recombining: for the sake of notational simplicity, we consider that this is part of EvalRound. The notation $\text{CtS}^\#$ stresses that CtS is run in low plaintext precision. In contrast, StC is run with a precision that corresponds to ε' , so that the output ciphertext is sufficiently accurate.

Algorithm 1: Overview of the bootstrapping algorithm in [33]; our bootstrapping algorithm follows the same high-level design.

Input : $\text{ct}_{\text{in}} = \text{Enc}(\vec{z}) \in \mathcal{R}_{q_0}^2$ with $\vec{z} \in \mathbb{C}^{N/2}$.

Output: $\text{ct}_{\text{out}} = \text{Enc}(\vec{z}') \in \mathcal{R}_q^2$ with $\vec{z}' \approx \vec{z}$ and $q \gg q_0$.

- 1 $\text{ct} \leftarrow \text{ModRaise}(\text{ct}_{\text{in}})$;
 - 2 $\text{ct}' \leftarrow \text{StC} \circ \text{EvalRound} \circ \text{CtS}^\#(\text{ct})$;
 - 3 $\text{ct}_{\text{out}} \leftarrow \text{ct} - \text{ct}'$;
 - 4 **return** ct_{out} .
-

Our proposed bootstrapping is identical to Algorithm 1, except that we give a different instantiation for EvalRound. For the new instantiation, we regard the ModRaise'd ciphertext $\text{ct} = \text{Enc}(q_0 \cdot \vec{I} + \Delta \cdot \vec{m})$ as a discrete CKKS encryption of $\vec{I}$ with scaling factor q_0 . In discrete CKKS, removing $\vec{e}$ from $q_0 \cdot \vec{I} + \vec{e}$ corresponds to the cleaning functionality. To clean $\vec{I}$, we use the following “decompose, clean, and recombine” strategy:

- (1) DigitExtract. We decompose a noisy integer $I = \sum_j I_j \beta^j + e$ for some β into noisy smaller digits $I_0 + e_0, I_1 + e_1, \dots \in [0, \beta)$. By adding K , the integer I may be assumed to be non-negative.
- (2) VecClean. We clean the digits in parallel, i.e., for every j , we transform $I_j + e_j$ into $I_j + e'_j$ with a bound on $|e'_j|$ that is much smaller than the bound on $|e_j|$.
- (3) Combine. We recombine the digits into an accurate version of I by computing $\sum_j (I_j + e'_j) \cdot \beta^j$.

The bootstrapping algorithm implements these steps homomorphically. We now formalize each step.

Definition 3.1 (Digit Extraction). Let $\beta > 1, \kappa > 0$ be integers. Let $\varepsilon \in (0, 1/8]$ and $\varepsilon_{\text{de}} \in (2\varepsilon, 1/4]$. Let ct be a discrete-CKKS ciphertext decrypting to integer vector $\vec{I} = (I_0, I_1, \dots, I_{N/2-1}) \in \{0, 1, \dots, \beta^\kappa - 1\}^{N/2}$ slots-encoded via the inclusion $\mathbb{Z} \subset \mathbb{C}$, and up to an error $\vec{e}$ with $\|\vec{e}\|_\infty \leq \varepsilon$. Digit extraction (DigitExtract) is a homomorphic operation that maps ct to $(\text{ct}_0, \text{ct}_1, \dots, \text{ct}_{u-1})$ such that, for all j and up to error $\vec{e}^{(j)}$ with $\|\vec{e}^{(j)}\|_\infty \leq \varepsilon_{\text{de}}$, the ciphertext ct_j decrypts to $\vec{I}^{(j)} = (I_{0j}, I_{1j}, \dots, I_{(N/2-1)j}) \in \{0, 1, \dots, \beta - 1\}^{N/2}$ where

$$I_k = \overline{I_{k(\kappa-1)} I_{k(\kappa-2)} \cdots I_{k0}} (\beta)$$

is the base- β representation of I_k , for all k . The I_{kj} 's are slots-encoded and can be represented as integers or as complex roots of unity.

Vectorized cleaning simply refers to cleaning individual digits in parallel. The key advantage of digit extraction comes from the fact that cleaning a smaller integer requires a smaller-degree cleaning polynomial and hence consumes less modulus.

Definition 3.2 (Vectorized Cleaning). Let $\beta > 1, \kappa > 0$ be integers and $\varepsilon_{\text{de}}, \varepsilon_{\text{clean}} \in (0, 1/4]$. Let $(\text{ct}_0, \text{ct}_1, \dots, \text{ct}_{\kappa-1})$ be a vector of ciphertexts such that, for all j and up to an error $\vec{e}^{(j)}$ with $\|\vec{e}^{(j)}\|_\infty \leq \varepsilon_{\text{de}}$, each ct_j decrypts to a vector in $\{0, 1, \dots, \beta - 1\}^{N/2}$ with integer or roots-of-unity discrete-CKKS representations. Vectorized cleaning (VecClean) is a homomorphic operation that maps $(\text{ct}_0, \text{ct}_1, \dots, \text{ct}_{\kappa-1})$ to $(\text{ct}'_0, \text{ct}'_1, \dots, \text{ct}'_{\kappa-1})$ where, for all j and up to an error $\vec{e}'^{(j)}$ with $\|\vec{e}'^{(j)}\|_\infty \leq \varepsilon_{\text{clean}}$, each ct'_j decrypts to the same plaintext as ct_j under the same discrete-CKKS representation. (Each map $\text{ct}_j \mapsto \text{ct}'_j$ is a valid discrete-CKKS cleaning.)

Recombination allows one to recover the input integer vector $\vec{I}$, but a significantly more precise version thereof. Its complexity depends on the chosen base β and the choice of discrete CKKS representations.

Definition 3.3 (Recombination). Let $\beta > 1, \kappa > 0$ be integers and $\varepsilon_{\text{clean}} \in (0, 1/4]$. Let $(\text{ct}_0, \text{ct}_1, \dots, \text{ct}_{\kappa-1})$ be a vector of ciphertexts such that, for all j and up to error $\vec{e}'^{(j)}$ with $\|\vec{e}'^{(j)}\|_\infty \leq \varepsilon_{\text{clean}}$, each ct_j decrypts to a vector $\vec{I}^{(j)} = (I_{0j}, I_{1j}, \dots, I_{(N/2-1)j}) \in \{0, 1, \dots, \beta - 1\}^{N/2}$ with integer or roots-of-unity discrete-CKKS representations. Recombination (Combine) is a homomorphic operation that maps $(\text{ct}_0, \text{ct}_1, \dots, \text{ct}_{\kappa-1})$ to ct' that, up to an error $\vec{e}'$ with $\|\vec{e}'\|_\infty \leq 2\beta^{\kappa-1} \cdot \varepsilon_{\text{clean}}$, decrypts to $\vec{I} = (I_0, I_1, \dots, I_{N/2-1}) \in \{0, 1, \dots, \beta^\kappa - 1\}^{N/2}$ slots-encoded via the inclusion $\mathbb{Z} \subset \mathbb{C}$, with $I_k = \sum_{0 \leq j < \kappa} \beta^j \cdot I_{kj}$ for all k .

Our algorithm is identical to Algorithm 1, with EvalRound instantiated as $\text{Combine} \circ \text{VecClean} \circ \text{DigitExtract}$.

THEOREM 3.4 (CORRECTNESS). Assume that ε' and $\varepsilon_{\text{clean}}$ satisfy $2\beta^{\kappa-1} \cdot \varepsilon_{\text{clean}} \leq \varepsilon'$. Given correct instantiations for digit extraction, vectorized cleaning and recombination, Algorithm 1 with EvalRound instantiated as $\text{Combine} \circ \text{VecClean} \circ \text{DigitExtract}$ is correct.

PROOF. As [33] provides a correctness analysis for Algorithm 1, it suffices to check that the composition $\text{Combine} \circ \text{VecClean} \circ \text{DigitExtract}$ is a correct instantiation of EvalRound, i.e., that it sends a ciphertext for a slots-encoded noisy vector $\vec{I} + \vec{e}$ with $\|\vec{e}\|_\infty \leq \varepsilon$ to a ciphertext for $\vec{I} + \vec{e}'$ with $\|\vec{e}'\|_\infty \leq \varepsilon'$.

As the computation proceeds in a SIMD manner (slots-wise), we focus on a single coordinate $I + e$ of $\vec{I} + \vec{e}$ to see what happens. Assuming that I belongs to $[0, \beta^\kappa)$, let $I = \overline{I^{(\kappa-1)} I^{(\kappa-2)} \cdots I^{(0)}} (\beta)$ denote the base- β decomposition of I . The digit extraction function splits $I + e$ into $I^{(0)} + e^{(0)}, I^{(1)} + e^{(1)}, \dots, I^{(\kappa-1)} + e^{(\kappa-1)}$, storing these in u different ciphertexts. The $e^{(j)}$'s may have increased compared to e but satisfy $\max_j |e^{(j)}| \leq \varepsilon_{\text{de}}$. Next, the cleaning step cleans each digit $I^{(0)}, I^{(1)}, \dots, I^{(\kappa-1)}$, changing the $e^{(j)}$'s into $e'^{(j)}$'s satisfying $\max_j |e'^{(j)}| \leq \varepsilon_{\text{clean}}$. Lastly, the recombination step allows

us to obtain a ciphertext for $I + e'$, where $|e'| \leq 2\beta^{\kappa-1} \cdot \varepsilon_{\text{clean}} \leq \varepsilon'$. This completes the proof. $\square$

Bootstrapping variants. Algorithm 1 is of the CtS-first type, as it finishes with StC. It was modified into an StC-first variant in [47] called EvalRound+ bootstrapping. The input ciphertext ct_{in} is at a higher level. One applies a high-precision StC on it, before ModRaise. The ModRaise'd ciphertext decrypts to $q_0 \cdot I + \Delta \cdot m$ (for coefficients encoding), for an integer I and the target plaintext m . This ciphertext then undergoes EvalRound $\circ$ CtS $^\#$, as in Algorithm 1, to obtain a high plaintext-precision slots-encoded ciphertext decrypting to $q_0 \cdot I$. As the ModRaise'd ciphertext is coefficients-encoded, the authors from [47] proposed to evaluate a high-precision CtS, in parallel of EvalRound $\circ$ CtS $^\#$, and, finally, take the difference of the ciphertexts obtained in the two branches (hence removing the term $q_0 \cdot I$ from $q_0 \cdot I + \Delta \cdot m$, in slots). EvalRound+ bootstrapping is oblivious to how EvalRound is instantiated, and hence it can be used with EvalRound instantiated as Combine $\circ$ VecClean $\circ$ DigitExtract. One drawback is that it involves two runs of CtS, one in small plaintext precision and one in high plaintext precision. However, the second one has ample modulus to be performed (all the modulus consumed by EvalRound $\circ$ CtS $^\#$), providing room to optimize its cost. On the plus side, EvalRound+ bootstrapping enjoys a faster StC as 1) it is performed with a smaller ciphertext modulus (it is StC-first) and 2) its plaintext precision needs to be sufficient to store $\Delta \cdot m$ as opposed to $q_0 \cdot I + \Delta \cdot m$.

Our algorithm can be optimized in the case of thin bootstrapping, i.e., when the plaintext has only n meaningful slots instead of $N/2$. Using a subring of $\mathcal{R}$ at the bottom level, of degree $N' \in [2n, N]$, it is known that StC and CtS can be accelerated (see, e.g., [19]). As a side-effect of using a subring, in Algorithm 1 and its EvalRound+ variant, the number of integers I to be cleaned is only $N' \leq N$. In our case, this can be exploited to accelerate vectorized cleaning. Indeed, in this case, we can pack the digits of the coefficients of I into $\lceil uN'/(N/2) \rceil$ ciphertexts. Packing and unpacking can be achieved using homomorphic rotations.

3.2 Extraction, Cleaning and Recombination

We now give approaches for implementing DigitExtract, VecClean and Combine. For the latter two algorithms, we only focus on the bit-decomposed ciphertext vectors. In Appendix A, we extend the vectorized cleaning and recombination to use trit-decomposed ciphertexts.

Digit extraction. The most direct approach to obtain the digit I_j of I consists of evaluating a Lagrange interpolation. If $|I_j|$ is known to be bounded by some integer $K = \beta^\kappa - 1$, then I_j can take only $2K + 1$ values. The Lagrange polynomial has degree $\leq 2K$, which may be implemented with multiplicative depth $\log(2K + 1)$. This provides a solution with low multiplicative depth, but it is typically numerically unstable when K is not very small, as the derivative of the interpolating polynomial on the distinguished points can be very high. To compensate, this forces us to instantiate CtS $^\#$ with a higher precision. To ensure a better numerical stability, one may consider a function that is constant in small intervals around the possible inputs and approximate it for the L_∞ norm on those intervals. Such a polynomial can be obtained through the improved

multi-interval Remez algorithm from [36]. The approximation becomes more precise when the degree increases and/or the intervals become thinner.

We now describe another approach. It starts from the observation that there exists an efficient procedure to extract bits from integers encoded as complex roots of unity [8]. We note that it generalizes to base β digits for $\beta \geq 2$. It takes as inputs that are close approximations to roots of unity of a power-of- β order $\{\exp(2i\pi I/\beta^\kappa) : 0 \leq I < \beta^\kappa\}$. As in the general case, obtaining the bits of I can be viewed as a Lagrange interpolation, but it turns out that the interpolating polynomials are sparse and can be evaluated efficiently (e.g., in base 2, the least significant bit is extracted using $x \mapsto (1 - x^{2^{\kappa-1}})/2$). As the input to DigitExtract is a ciphertext whose plaintext is an integer vector that is slots-encoded via the inclusion $\mathbb{Z} \subset \mathbb{C}$, to use this algorithm, we first need to transform it into a ciphertext for the same integer vector but with roots of unity representation. For this purpose, we first evaluate $x \mapsto \exp(2i\pi x/\beta^\kappa)$. We approximate the complex exponential by a polynomial $P_{\text{exp}} = P_{\text{cos}} + iP_{\text{sin}}$, where P_{cos} and P_{sin} are polynomials that respectively approximate scaled versions of cos and sin. As we are interested in only a few input points and we tolerate relatively large output inaccuracy (as extraction is followed by cleaning anyway), it turns out in practice that a degree-8 polynomial suffices.

Digit cleaning. Cleaning bits was introduced in [26]: as recalled in Lemma 2.1, applying the h_1 map essentially doubles the precision of the bit under scope. It can be evaluated within two multiplicative levels. One option for VecClean is to iterate the evaluation of the h_1 map several times, doubling the accuracy every time, until matching the target accuracy. Since we have low precision at the beginning, it suffices to use a very small scaling factor Δ . Then, between two iterations, the scaling factor Δ can be squared, so that the homomorphic noise does not pollute the accuracy gain from h_1 . Another option for cleaning bits would be to use a higher degree polynomial which still sends $x \in \{0, 1\}$ to x but has several successive derivatives that vanish in both 0 and 1. This may decrease the total number of multiplicative levels, allowing reaching a desired accuracy, but it has two drawbacks: first, evaluating higher degree polynomials is more costly; second, it seems that the final scaling factor is required for all multiplicative levels, in contrast with the iterative method that uses such a scaling factor only at the last iteration. The latter cancels the multiplicative depth advantage.

Recombination. The recombination step, consisting of $(\kappa - 1)$ sequential integer multiplications and additions, can be efficiently performed without any modulus consumption.

Cost analysis. The most time-consuming homomorphic operations during polynomial evaluation are ciphertext-ciphertext, or non-scalar, multiplications. Note that the direct approach requires the evaluation of κ different polynomials on the same input. It can be achieved with $2\sqrt{\kappa d}$ non-scalar multiplications, where $d \geq \beta^\kappa$ denotes the degree of the polynomials, using the Paterson-Stockmeyer algorithm. In contrast, the digit extraction polynomials from complex roots of unity can be seen as polynomials of degree $\beta^\kappa, \beta^{\kappa-1}, \dots, \beta$ in $x, x^\beta, \dots, x^{\beta^{\kappa-1}}$, respectively. As the degrees follow a geometric series, this results in $O(\sqrt{\beta^\kappa})$ non-scalar multiplications for that approach.

Therefore, for the same $K = \beta^K - 1$, the indirect approach is asymptotically faster than the direct one, with $O(\sqrt{K})$ non-scalar multiplications versus $O(\sqrt{K} \cdot \log K)$ for the direct one. On the other hand, the direct one consumes less modulus. In practice, when K is small, the difference in performance is not very significant, and the direct approach will outperform in terms of throughput. Additionally, if there is little modulus left for computations, saving modulus with the direct approach can have a significant impact on throughput. However, if K increases or there is enough modulus left for computation, the indirect approach will become superior.

The digit cleaning and the recombination costs are linear in the digit length κ . Concretely, $O(\kappa \cdot \text{nb_cln})$ scalar multiplications for h_1 -evaluations, and $O(\kappa)$ integer multiplications for recombination, where nb_cln is the number of iterations for digit cleaning.

Finally, we analyze the asymptotic dependency of the secret key Hamming weight h . When increasing h , the distribution of the “I” term can be modeled as a Gaussian with standard deviation $\sqrt{h}$, so K grows as $O(\sqrt{h})$. The number of digits, κ , to be considered grows as $\log_2 K \approx 0.5 \cdot \log_2 h$. We hence need $0.5 \cdot \log_2 h$ ciphertexts at the end of extraction. For the extraction itself, the Lagrange interpolation from roots of unity has degree K . Using Paterson-Stockmeyer, the cost is $O(\sqrt{K}) = O(h^{1/4})$ scalar multiplications.

3.3 Modulus Consumption

We now discuss the modulus consumption of the bootstrapping algorithm. This directly impacts throughput, as the total amount of available modulus is bounded (else the Ring-LWE parametrization becomes insecure), and we are interested in maximizing the number of homomorphic multiplications between consecutive bootstraps.

The ring degree N impacts the modulus consumption, as every multiplicative level comes with a homomorphic computation noise whose bit-length is $O(\log N)$. This contributes to a total of $O(\log N)$ modulus consumption. Now, let us consider the modulus consumption dependency in the target plaintext precision t (i.e., the precision of the input and output of bootstrapping).

- The precision before cleaning can be bounded independently of t , as cleaning allows to reach any precision, from inputs that are barely precise enough so that the underlying plaintexts are well-defined. The overall modulus consumption in all these steps is hence $O(1)$ (assuming that N is constant). Let t_0 be the plaintext precision right before cleaning.
- Assume that we use the same cleaning function at every iteration and that it improves the precision by a factor $a > 1$ in b multiplicative levels. The first iteration is performed at plaintext precision $t_0 \cdot a$ as it aims to multiply precision by a factor a . It hence consumes $t_0 \cdot a \cdot b$ bits of modulus. The modulus consumption of subsequent iterations follows a geometric progression; the total modulus consumption is then

$$t_0 \cdot a \cdot b + t_0 \cdot a^2 \cdot b + \dots + t \cdot b \approx t \cdot \frac{ab}{a-1} = O(t) \quad .$$

- If using d levels of DFT recursion (typically, we have $d \leq 3$), then the modulus consumption of StC is $\approx d \cdot t$.

Note that only the end of cleaning and StC have multiplicative levels with large scaling factors.

Comparison with prior approaches. The direct approach is to take the classical CKKS bootstrapping [14, 19] and increase the precision of every multiplicative level linearly with t . Further, this algorithm relies on a polynomial approximation to a trigonometric function, whose degree must grow as $O(t)$ so that it is accurate enough. This leads to a multiplicative depth growing as $O(\log t)$ and a modulus consumption that grows as $O(t \log t)$. The running time also grows faster than that of our algorithm, due to the increased number of multiplicative levels.

Finally, we discuss the Meta-BTS algorithm from [6], which consists in running a precision t_0 bootstrapping t/t_0 times. Assuming that t_0 is a constant, this gives $O(t)$ iterations. The modulus consumption still grows linearly in $O(t)$, as this is the precision of the input and output. The constant is lower than ours, though.

4 Low Modulus-Consumption Cleaning

Although starting from small precision and iteratively cleaning the digits reduces modulus consumption greatly, the precision-increasing cleaning steps may consume significant modulus. In this section, we propose techniques to further reduce the modulus consumption, saving a factor 4 compared with the direct approach.

4.1 Low Modulus Consumption for Bits

We first consider the degree-2 cleaning polynomial $h_1 : x \mapsto 3x^2 - 2x^3$ for bits $\{0, 1\}$. We are given as input a ciphertext $\text{ct} \in \mathcal{R}_Q^2$ for some ciphertext modulus Q , decrypting to a vector $\vec{z} = \vec{b} + \vec{e}$ of noisy bits (in slots), with $\vec{b} \in \{0, 1\}^{N/2}$ and $\|\vec{e}\|_\infty \leq 2^{-t}$ for some $t \geq 0$. The scaling factor Δ is a little more than 2^t , so that the homomorphic computation error is not larger than the bound on $\vec{e}$. The purpose of h_1 is to obtain a ciphertext $\text{ct}_{\text{out}} \in \mathcal{R}_{Q_{\text{out}}}^2$ for $\vec{z}_{\text{out}} = \vec{b} + \vec{e}_{\text{out}}$, for the same $\vec{b}$ but with $\|\vec{e}_{\text{out}}\|_\infty \leq 2^{-2t+c}$ for some small constant c (see Lemma 2.1). The scaling factor of the output ciphertext ct_{out} hence needs to be of the order of Δ^2 . Our goal is to reduce the modulus consumption, i.e., the ratio Q/Q_{out} .

The black-box approach. We start with a straightforward method that treats homomorphic addition and multiplication as black boxes. This serves as a baseline commonly considered in the literature. Given $\text{ct} \in \mathcal{R}_Q^2$, the direct evaluation of h_1 would proceed as Figure 2 (left side). At Step 1, multiplying the ciphertext by Δ has the effect of increasing the scaling factor to Δ^2 , while keeping a plaintext that is $\approx \vec{z}$. At Steps 2 and 3, the homomorphic multiplications are with a scaling factor Δ^2 , leading to a loss of a factor Δ^2 in ciphertext modulus (twice). Note that Q/Δ^2 and Q/Δ^4 may not be integers. We abuse notations to refer to integers that are close to these values. Finally, the multiplications by 3 and 2 of Step 4 are plaintext-ciphertext multiplications with an exact and small plaintext, for which no rescaling is required. Overall, the modulus consumption of this black-box approach is Δ^4 .

Inverse rescaling approach. As a first non-black-box optimization, we observe that multiplying by Δ in Step 1 of the black-box approach actually gives us a ciphertext ct_1 with modulus $Q\Delta$.³ Up to adjusting the ciphertext moduli accordingly, the rest of the

³This observation may be considered folklore; however, we make it explicit here for clarity and comparison.

1: $ct_1 := \Delta \cdot ct \in \mathcal{R}_Q^2$;	1: $ct_1 := \Delta \cdot ct \in \mathcal{R}_{Q\Delta}^2$;	1: $ct_1 := ct \otimes ct \in \mathcal{R}_Q^3$;
2: $ct_2 := \text{Mult}_{\Delta^2}(ct_1, ct_1) \in \mathcal{R}_{Q/\Delta^2}^2$;	2: $ct_2 := \text{Mult}_{\Delta^2}(ct_1, ct_1) \in \mathcal{R}_{Q/\Delta}^2$;	2: $ct_2 := \text{Relin}(ct_1) \in \mathcal{R}_Q^2$;
3: $ct_3 := \text{Mult}_{\Delta^2}(ct_1, ct_2) \in \mathcal{R}_{Q/\Delta^4}^2$;	3: $ct_3 := \text{Mult}_{\Delta^2}(ct_1, ct_2) \in \mathcal{R}_{Q/\Delta^3}^2$;	3: $ct_3 := \text{Relin}(ct \otimes ct_2) \in \mathcal{R}_Q^2$;
4: $ct_{\text{out}} := 3 \cdot ct_2 - 2 \cdot ct_3 \in \mathcal{R}_{Q/\Delta^4}^2$.	4: $ct_{\text{out}} := 3 \cdot ct_2 - 2 \cdot ct_3 \in \mathcal{R}_{Q/\Delta^3}^2$.	4: $ct_4 := 3 \cdot \Delta \cdot ct_2 - 2 \cdot ct_3 \in \mathcal{R}_Q^2$;
		5: $ct_{\text{out}} := \text{Rescale}_{\Delta}(ct_4) \in \mathcal{R}_{Q/\Delta}^2$.

Figure 2: The black-box approach in the literature (left), the inverse rescaling approach (middle), and the thrifty approach (right) for evaluating h_1 .

algorithm stays the same. This observation reduces modulus consumption from Δ^4 to Δ^3 . See Figure 2 (center).

The thrifty approach. We improve the modulus consumption even further, by a detailed inspection and optimization of the components of homomorphic multiplication. Homomorphic multiplication Mult_{Δ} for a scaling factor Δ proceeds in three steps:

- **Tensor:** given $ct = (a, b)$ and $ct' = (a', b')$ in $\mathcal{R}_Q^2$, tensor outputs $ct \otimes ct' = (aa', ab' + a'b, bb') \in \mathcal{R}_Q^3$; if ct and ct' respectively decrypt to $\bar{z}$ and $\bar{z}'$ under the key $(s, 1)$ and with scaling factor Δ , then the output decrypts to $\bar{z} \odot \bar{z}'$ with scaling factor Δ^2 under $(s^2, s, 1)$ where $\odot$ refers to the component-wise product over $\mathbb{C}^{N/2}$.
- **Relinearization:** given $ct \in \mathcal{R}_Q^3$, Relin outputs $ct' \in \mathcal{R}_Q^2$ with the same plaintext (up to a homomorphic evaluation error) and the same scaling factor.
- **Rescaling:** given $ct \in \mathcal{R}_Q^2$, Rescale outputs $ct' \in \mathcal{R}_{Q/\Delta}^2$ with the same plaintext (up to a homomorphic evaluation error) and a scaling factor divided by Δ .

In the thrifty approach, we first view the input ciphertext $ct = (a, b) \in \mathcal{R}_Q^2$ as an error-free ciphertext decrypting to $\bar{z} \in \mathbb{C}^{N/2}$, i.e., satisfying:

$$\text{DFT} \circ \text{Dec}(ct) = \Delta \cdot \bar{z} ,$$

where Dec consists in taking the inner product with $(s, 1)$. As the discrete Fourier transform DFT between $\mathbb{R}[X]/(X^N + 1)$ and $\mathbb{C}^{N/2}$ is a homomorphism, the tensor operation does not introduce any error:

$$\text{DFT} \circ \text{Dec}'(ct \otimes ct) = \Delta^2 \cdot \bar{z} \odot \bar{z} ,$$

where Dec' refers to the inner product with $(s^2, s, 1)$. Based on this observation, we evaluate h_1 as described in the right-hand side of Figure 2. The modulus consumption is reduced from Δ^4 or Δ^3 , down to Δ .

THEOREM 4.1. *The thrifty approach given in Figure 2 is correct.*

PROOF. To prove correctness, we examine the errors underlying the ciphertexts occurring during the computation. Recall that relinearization and rescaling introduce homomorphic evaluation errors, which are bounded independently of the scaling factor. The input satisfies $\text{DFT} \circ \text{Dec}(ct) = \Delta \cdot \bar{z} \in \mathbb{C}^{N/2}$. As we have seen, the ciphertext ct_1 satisfies $\text{DFT} \circ \text{Dec}'(ct_1) = \Delta^2 \cdot \bar{z} \odot \bar{z}$. We may write

$$\text{DFT} \circ \text{Dec}(ct_2) = \Delta^2 \cdot \bar{z} \odot \bar{z} + \vec{e}_{\text{Relin}} ,$$

for some relinearization error e_{Relin} . At Step 3, we first apply tensor and relinearization. We have

$$\text{DFT} \circ \text{Dec}(\text{Relin}(ct_3)) = (\Delta \cdot \bar{z}) \odot (\Delta^2 \cdot \bar{z} \odot \bar{z} + \vec{e}_{\text{Relin}}) + \vec{e}'_{\text{Relin}} ,$$

for some other relinearization error $\vec{e}'_{\text{Relin}}$. Therefore, at Step 4, we have:

$$\text{DFT} \circ \text{Dec}(\text{Relin}(ct_4)) = \Delta^3 \cdot h_1(\bar{z}) + \vec{e}_4 ,$$

where h_1 is applied component-wise and $\|\vec{e}_4\|_{\infty}$ is at most of the order of Δ . After rescaling by Δ , we obtain

$$\text{DFT} \circ \text{Dec}(ct_{\text{out}}) = \Delta^2 \cdot h_1(\bar{z}) + \vec{e}_{\text{out}} ,$$

where $\|\vec{e}_{\text{out}}\|_{\infty}$ is $O(1)$ (as a function of Δ). Lemma 2.1 then allows us to conclude. $\square$

We now consider the modulus consumption resulting from k cleaning iterations implemented with the thrifty approach.

THEOREM 4.2. *Assume we apply the thrifty approach as described in Figure 2, k times, to increase the scaling factor from Δ to Δ^2 , to Δ^4 , etc up to Δ^{2^k} . The overall modulus consumption is Δ^{2^k-1} .*

PROOF. We observe that at the j th iteration, the scaling factor increases from $\Delta^{2^{j-1}}$ to Δ^{2^j} , resulting in a modulus consumption of $\Delta^{2^{j-1}}$. Then the total modulus consumption is $\prod_{1 \leq j \leq k} \Delta^{2^{j-1}} = \Delta^{\sum_{1 \leq j \leq k} 2^{j-1}} = \Delta^{2^k-1}$. $\square$

Asymptotically, we double the precision at every cleaning step, and the scaling factor should grow from Δ to Δ^2 . However, in practice, there is a gap of c bits between $\log \Delta$ and the plaintext precision t , due to homomorphic evaluation errors (e.g., $c \approx 12$). When applying cleaning, we double $\log \Delta$, but this gap is also doubled from c to $2c$, even though the homomorphic evaluation errors still have only around c bits: the resulting ciphertext has a scaling factor Δ^2 and precision $2t$. To handle this, we reduce the scaling factor, using a rescaling operation, so that the gap is reduced from $2c$ to c , because otherwise the scaling factor would grow faster than the precision and would result in unnecessary modulus consumption in the subsequent cleaning steps. Therefore, we rescale the output ciphertext by $\Delta/2^t$, consuming $\Delta^2/2^t$ bits in total.

A further difficulty comes from the fact that we need to scale z^2 (in ct_2) by Δ at Step 4, so that the scaling factor matches with that of z^3 (in ct_3). This implicitly assumes that Δ is an integer. If this is not the case, then it is tempting to encode Δ as a plaintext, but this creates a homomorphic evaluation error that is too large for the correctness proof to work. Instead, if Δ is not an integer, we replace

it by a close-by integer Δ' at the outset. Consider $\Delta' = \lfloor \Delta \rfloor \in \mathbb{Z}$. Then we may write

$$\text{DFT} \circ \text{Dec}(\text{ct}) = \Delta \cdot \vec{z} = \Delta' \cdot \vec{z}'$$

for $\vec{z}' = (\Delta/\Delta')\vec{z} \in \mathbb{C}^{N/2}$. Here we observe that

$$\|\vec{z}' - \vec{z}\|_\infty = \left\| \left(\frac{\Delta' - \Delta}{\Delta} \right) \vec{z} \right\|_\infty \leq \frac{\|\vec{z}\|_\infty}{\Delta}.$$

Hence, we may view $\vec{z}'$ as a new representation of the initial vector $\vec{z}$, with an integer scaling factor Δ' .

4.2 Extensions

The approach can be further generalized. Let $\text{ct} \in \mathcal{R}_Q^2$ be a ciphertext decrypting to $\vec{z} \in \mathbb{C}^{N/2}$, with scaling factor Δ . Based on the fact that the tensor operation is error-free, we may perform an error-free high-degree (integer-coefficient) polynomial evaluation. Assume we want to evaluate $x \mapsto \sum_{0 \leq j \leq k} \alpha_j x^j$ with integer coefficients α_j . We can proceed as follows.

- For $2 \leq j \leq k$, compute $\text{ct}_j = \Delta^{k-j} \cdot \text{Relin}(\text{ct}^{\otimes j}) \in \mathcal{R}_Q^2$, where the notation $\text{ct}^{\otimes j}$ means that we tensor j copies of ct ; this decrypts to $\Delta^k \cdot \vec{z}^j + \vec{e}_{\text{relin}}$ with $\|\vec{e}_{\text{relin}}\| \leq O(\Delta^{k-j})$.
- Compute and return $\text{ct}_{\text{out}} = \alpha_0 + \alpha_1 \cdot \text{ct} + \sum_{2 \leq j \leq k} \alpha_j \text{ct}_j$.

The scaling factor of ct_{out} is Δ^k . As the terms α_0 and $\alpha_1 \cdot \text{ct}$ are exact, the largest relinearization error is $O(\Delta^{k-j_0})$, where $j_0 = \min\{j \geq 2 : \alpha_j \neq 0\}$. In the case of h_1 , we have $k = 3$ and $j_0 = 2$, leading to a Δ^2 -gap between “plaintext” and error.

We detail the thrifty approach for cleaning trits in Appendix A. Moreover, it can be applied to all the cleaning polynomials in Table 3 by considering the total degree rather than the individual degree for those polynomials that involve conjugations.

5 Experiments

Our code is developed upon the C++ HEaAn library [25], using the grafting technique from [17]. All our experiments were performed using a single thread on an Apple M4 Pro equipped with 32GiB of RAM, running macOS Sequoia 15.2. All our experiments were repeated at least 20 times. Reported execution times are averaged over experiments. Precision is defined as the negative base-2 logarithm of the maximum error shown when decrypted, observed across all experiments.

5.1 Arithmetic Aspects

Grafting. Our implementation uses the grafting technique proposed in [17]. It removes the dependencies between the ciphertext modulus and the scaling factors that exist in RNS-CKKS. During iterative cleanings, we start with a ciphertext with small precision and increase the precision by cleaning with the increased scaling factors. Grafting avoids situations where a large number of small-size NTT primes are required but there are not enough of them. In grafting, two ciphertext moduli are usually not divisible by one another, so the rescaling operations are performed using modulus switching. When rescaling from a ciphertext modulus $Qa \in \mathbb{Z}$ to another ciphertext modulus $Qb \in \mathbb{Z}$, we refer the situation to rescaling by $\lceil \log_2(a/b) \rceil$ bits where $a/b \approx 2^{\lceil \log_2(a/b) \rceil}$, which is usually the case. Also, the scaling factors are not directly tied to the

modulus and are changing during the operations; therefore, they should be accurately tracked along the computation.

High-Precision Arithmetic. The scaling factors must be large enough to achieve high-precision bootstrapping, as the inherent encoding and decoding errors are roughly inversely proportional to the scaling factors. Also, we must handle scaling factors with large precision while tracking them throughout homomorphic computations and using them for encoding and decoding. To this end, we use libquadmth, the GCC library for quadruple precision arithmetic. Note that the CKKS ciphertexts do not need high-precision real numbers as they are represented exactly using integers.

5.2 Implementation Details

Security and Failure Probability. All of our parameters satisfy 128-bit security for the Ring-LWE instances they rely upon, according to the lattice estimator [3]. In addition, we also target more advanced security notions like IND-CPA-D and circuit privacy, as well as secure Threshold-FHE, and thus require bootstrapping to succeed with a probability that is exponentially close to 1 [18]. Our parameter sets extract 5 bits from the integer approximating the rounding function for the erroneous integer $I + (\Delta/q_0)m$ for a 32-period piecewise input domain, a union of small intervals centered at each integer $I \in [-16, 15]$. Using a Hamming weight $h = 30$ for the ternary secret key, we have a zero failure probability for this approximation, thanks to $\|I + (\Delta/q_0)m\|_\infty = \|(b + as)/q_0\|_\infty \leq (h + 1)/2 = 15.5$, where $(a, b) \in \mathcal{R}_{q_0}$ is the ciphertext before ModRaise.

Extractions. We implement both extractions, the direct approximation of integers to bits, and the indirect approximation through roots of unity.

In the indirect case, as explained in Section 3.2, we first use an approximation of the complex exponential function, to map erroneous integers $I \in [-2^{K-1}, 2^{K-1} - 1]$ to erroneous roots of unities $\exp(2\pi I/2^K)$. For this purpose, we use a degree-8 dense polynomial, where the inputs are scaled into $[-\pi, \pi]$. Note that the degree-8 polynomial can be computed in $\log_2 8 = 3$ multiplicative levels instead of $\lfloor \log_2 8 + 1 \rfloor = 4$, thanks to grafting: we evaluate the polynomial made of the first 8 monomials using the scaled coefficients, add x^8 , and then reset the scaling factor by dividing it by the highest-degree coefficient. From the roots of unity, we obtain bits by relying on the bit extraction technique of Bae et al. [8]. Other choices are possible for the complex exponential polynomial, such as with degrees 4 and 16, which can be evaluated in multiplicative depths of 2 and 4, respectively. The polynomials of degrees 4, 8 and 16 give approximation precisions of 3.3, 12 and 36.5 bits, respectively. We choose degree 8, as it is accurate enough to compute the complex exponential of the integer I while preserving its precision. Note that the initial accuracy of I (compared to $\lceil I \rceil$) is roughly an inverse of the gap q_0/Δ since we have $I + (\Delta/q_0)m$ in the coefficients after ModRaise.

In the case of direct extraction, we implement five different degree-47 polynomials, one for each bit. This degree allows us to extract bits and preserves precision. Degree-47 is a good compromise between ease of evaluation and small approximation error. The evaluation of these five polynomials requires only 22 non-scalar multiplications since we can precompute the odd polynomial basis up to degree 15 (8 terms) and the Chebyshev polynomials

$T_2, T_4, T_8, T_{16}, T_{32}$ (5 terms), and reuse them very efficiently: evaluating an odd polynomial $p(X)$ of degree 47 only requires 2 non-scalar multiplication by expressing it as $p(X) = p_0(X) + T_{16} \cdot p_1(X) + T_{32} \cdot p_2(X)$, while using multiplicative depth of 6. We remark that the indirect digit extraction via roots of unity also requires a similar cost of 21 key-switchings, but using two more multiplicative depths: 6 non-scalar multiplications for mapping the integers to roots of unity, 10 non-scalar multiplications, and 5 conjugations for extracting digits from roots of unity.

Iterative Cleaning. For cleaning the extracted bits, we use $h_1(x) = 3x^2 - 2x^3$. The cleaning polynomial is evaluated iteratively using the optimization techniques explained in Section 4. As the precision doubles with each iteration, a larger initial precision requires fewer cleaning iterations. However, the initial precision is limited by the scaling factors of the homomorphic DFTs, i.e., StC and CtS, and digit extraction. Increasing them would noticeably increase the modulus consumption.

Selecting Scaling Factors and Modulus Gap. We can achieve a given target bootstrapping precision by appropriately choosing the scaling factors for each sub-operation and the bottom modulus. As mentioned above, the gap $g = q_0/\Delta$ for the StC scaling factor Δ plays a crucial role by setting the initial precision of the erroneous integer, along with the scaling factors for StC and CtS. When the erroneous bits are extracted, their precision cannot exceed the gap. In practice, the Most Significant Bit (MSB) loses around 4 bits of precision. During each cleaning iteration, the precision is roughly doubled. When the bits are combined and subtracted with the high-precision CtS result, we get the messages divided by the gap. Thus, the precision of the message, when scaled correctly, is $(\log g)$ -bit smaller than that of the combined integer. More precisely, let us assume we have high-precision CtS result $\tilde{I} := I + (\Delta/q_0)m + \epsilon_0$ and the EvalRound result $\lceil \tilde{I} \rceil + \epsilon_1$ for small errors ϵ_0, ϵ_1 of magnitude 2^{-P} . The subtraction gives $(\Delta/q_0)m + \epsilon_0 - \epsilon_1$, where the magnitude of the relative error becomes $\approx 2^{-(P - \log g)}$. The resulting bootstrapping precision becomes $\leq (2^{\text{nb_cln}} \cdot (\log g - 4) - \log g)$ bits, where nb_cln is the number of iterations for cleanings.

Note that the scaling factors can be chosen based on the precisions required for each step, heuristically considering the inherent error term in the ciphertext for each operation. When decrypted and decoded, the RLWE error is multiplied by the secret key and the decoding matrix, introducing a $O(\sqrt{N}h)$ error to the message, where h is the secret key Hamming weight and N is the ring degree. Due to this inherent noise, the scaling factor should be at least $\delta \approx \log \sqrt{N}h$ bits larger than the message precision; hence, the smallest possible scaling factor Δ for a given plaintext precision t would satisfy $\log \Delta \gtrsim t + \delta$. Especially during iterative cleanings, the scaling factors $\Delta \approx 2^{t+\delta}$ and precision t , respectively, change into $\Delta' \approx \Delta^3 / (\Delta \cdot 2^\delta) \approx 2^{2t+\delta}$ and $t' = 2t$, consuming $(t + 2\delta)$ bits of modulus.

StC-First Versus CtS-First. The EvalRound bootstrapping algorithm [33] (see Algorithm 1) starts with CtS, whereas the EvalRound+ algorithm [47] allows to start with StC. We opt for the latter as it allows to decrease modulus consumption in StC, for a moderate increase in execution time.

Our FGb Variant. To compare with our bootstrapping, we choose one of the standard and well-optimized FHE parameter sets with ring degree $N = 2^{16}$, namely the FGb of [25]. It is similar to a parameter set in the Lattigo library [35]. We consider a modification of FGb so that it incorporates grafting [17]. For fair comparison, we made the following modifications: using secret key encapsulation to decrease the degree of the EvalMod polynomial and to decrease the bootstrapping failure probability; using only two multiplicative levels for StC instead of three to lower modulus consumption while not affecting the execution time too much; using a larger dnum of 6 instead of 5, maximizing the throughput among the possible choices. We refer to this modified FGb parameter set as FGb*.

5.3 Experimental Results and Comparison

In Table 2, we introduce our parameters, their modulus consumption, and timings with different numbers of cleaning iterations. The ring dimension is $N = 2^{16}$, and the maximum modulus PQ is set to 1533 bits. We propose parameter sets for precisions near 20, 40 and 80, based on direct extraction (Direct) and extraction via roots of unity (Indirect). The number of cleanings is given in the parameter names, followed by the precision of the bootstrapping.

For EvalRound+, the figures are not based on our code but borrowed from [47]. As the timings from [47] are unfairly uncompetitive due to a different code-base, we did not report them in Table 2. We expect that using sparse secret encapsulation, the EvalRound+ will also have a very small failure probability (for a small modulus consumption increase).

For any ring degree (here $N = 2^{16}$), there is a maximum modulus budget that can be used for RLWE samples to achieve 128-bit security. When the rank of the gadget decomposition for the switching keys is set, the maximum ciphertext modulus is determined, which is shown as the budget in Table 2. EvalRound+ has a larger budget than the other parameter sets, possibly thanks to a larger gadget rank dnum. We chose $\text{dnum} = 6$, the same as FGb*’s.

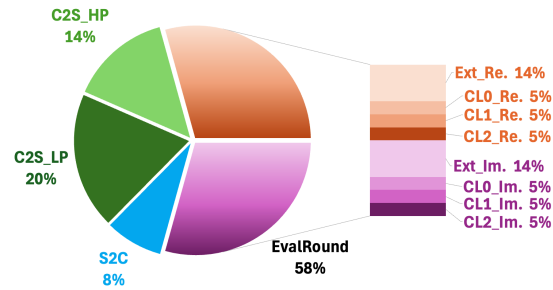
Throughput Comparison. To compare the bootstrapping algorithms, we consider throughput. It is defined as the remaining available modulus after bootstrapping divided by the bootstrapping time. The direct parameter set Direct3Cln81 with 3 cleanings provides a throughput of $494/20.88 \approx 23.66$ bits/s. We compare it with Meta-BTS [6] applied to the FGb* parameter set. By iterating the FGb* bootstrapping four times, consuming an extra $80 - 20 = 60$ bits of modulus for storing the plaintext to be bootstrapped, we can obtain ≈ 80 bits of precision by repeating the FGb* bootstrapping four times sequentially. This provides roughly an 80-bit precision CKKS bootstrapping that runs in approximately $8.51 \cdot 4 = 34.04$ seconds. In terms of throughput, this gives 14.39 bits/s, which is $1.64\times$ smaller than with our parameter set Direct3Cln81.

Profiling Sub-procedures. We provide the detailed modulus consumption during bootstrapping in Figure 1 and the run-time in Figure 3. In Figure 1, the block on the right is for the high-precision CtS with 114-bit scaling factors. In Figure 3, the blocks on the right are for EvalRound, including extractions and cleanings for both the real and imaginary parts.

As cleaning is done in multiple parallel tracks, it consumes a significant part of the total execution time. The two CtS’s with both lower and high precision are next, in terms of run-time share. We

Table 2: Modulus consumptions and execution times for various CKKS bootstrapping algorithms. Note that the throughput should be compared between the parameter sets with similar bootstrapping precision.

Parameter Sets ($N = 2^{16}$)	Cleaning iterations	Prec.	Budget	Bottom	BTS modulus consumption			Available modulus	Time (s)	Throughput (modulus/time)
					StC	ER/EM	CtS# (CtS)			
Indirect2Cln17	2	17	1289	44	62	321	132	670	12.6	53.09
Direct2Cln19	2	19	1289	46	66	300	135	742	14.3	51.78
Indirect3Cln37	3	37	1289	70	114	473	84 (210)	632	18.4	34.29
Direct2Cln38	2	38	1289	79	116	349	105 (300)	640	17.9	35.75
Indirect4Cln80	4	80	1289	110	194	457	84 (354)	443	20.7	21.42
Direct3Cln81	3	81	1289	112	188	402	93 (354)	494	20.9	23.66
FGb*	-	21	1289	58	58	464	159	550	8.5	64.63
FGb*-Meta-BTS	-	≈40	1289	78	58	464	159	530	17.0	31.14
FGb*-Meta-BTS	-	≈80	1289	118	58	464	159	490	34.0	14.39
EvalRound+	-	16	1298	58	80	522	93	546	-	-

**Figure 3: Profiled run-time of Parameter Set Direct3Cln81.**

observe that the high-precision CtS is faster than the low-precision CtS: this is because the high-precision CtS is run at a smaller modulus. The 2-level StC is about 10% of the total execution time, which is of a similar portion compared to usual CKKS bootstrapping. Those proportions do not change with other parameter sets.

Acknowledgments

Part of this work was conducted while Hyeonmin Choe was with Seoul National University, and Jaehyung Kim and Elias Suvanto were with CryptoLab Inc. This work was supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korean government (MSIT) (No. RS-2025-02217656). Jaehyung Kim acknowledges support from the Stanford Graduate Fellowship and DARPA. Opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of DARPA.

References

- [1] Rashmi Agrawal, Jung Ho Ahn, Flavio Bergamaschi, Ro Cammarota, Jung Hee Cheon, Fillipe D. M. de Souza, Huijing Gong, Minsik Kang, Duhyeon Kim, Jongmin Kim, Hubert de Lassus, Jai Hyun Park, Michael Steiner, and Wen Wang. 2023. High-precision RNS-CKKS on fixed but smaller word-size architectures: theory and application. In *Proceedings of the 11th Workshop on Encrypted Computing & Applied Homomorphic Cryptography (WAHC '23)*. Association for Computing Machinery, New York, NY, USA, 23–34. doi:10.1145/3605759.3625257
- [2] Ahmad Al Badawi, Jack Bates, Flavio Bergamaschi, David Bruce Cousins, Saroja Erabelli, Nicholas Genise, Shai Halevi, Hamish Hunt, Andrey Kim, Yongwoo Lee, Zeyu Liu, Daniele Micciancio, Ian Quah, Yuriy Polyakov, Saraswathy R.V., Kurt Rohloff, Jonathan Saylor, Dmitriy Sponitsky, Matthew Triplett, Vinod Vaikuntanathan, and Vincent Zucca. 2022. OpenFHE: Open-Source Fully Homomorphic Encryption Library. In *Proceedings of the 10th Workshop on Encrypted Computing & Applied Homomorphic Cryptography (WAHC '22)*. Association for Computing Machinery, New York, NY, USA, 53–63. doi:10.1145/3560827.3563379
- [3] Martin R. Albrecht, Rachel Player, and Sam Scott. 2015. On the concrete hardness of Learning with Errors. *Journal of Mathematical Cryptology* 9, 3 (2015), 169–203. doi:10.1515/jmc-2015-0016
- [4] Andreea Alexandru, Andrey Kim, and Yuriy Polyakov. 2025. General Functional Bootstrapping Using CKKS. In *Advances in Cryptology – CRYPTO 2025*. Springer-Verlag, Berlin, Heidelberg, 304–337.
- [5] Gilad Asharov, Abhishek Jain, Adriana López-Alt, Eran Tromer, Vinod Vaikuntanathan, and Daniel Wichs. 2012. Multiparty Computation with Low Communication, Computation and Interaction via Threshold FHE. In *Advances in Cryptology – EUROCRYPT 2012*. Springer, Berlin, Heidelberg, 483–501.
- [6] Youngjin Bae, Jung Hee Cheon, Wonhee Cho, Jaehyung Kim, and Taekyung Kim. 2022. META-BTS: Bootstrapping Precision Beyond the Limit. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (Los Angeles, CA, USA) (CCS '22)*. Association for Computing Machinery, New York, NY, USA, 223–234. doi:10.1145/3548606.3560696
- [7] Youngjin Bae, Jung Hee Cheon, Jaehyung Kim, and Damien Stehlé. 2024. Bootstrapping Bits with CKKS. In *Advances in Cryptology – EUROCRYPT 2024*. Springer Nature Switzerland, Cham, 94–123.
- [8] Youngjin Bae, Jaehyung Kim, Damien Stehlé, and Elias Suvanto. 2025. Bootstrapping Small Integers With CKKS. In *Advances in Cryptology – ASIACRYPT 2024*. Springer Nature Singapore, 330–360.
- [9] Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter M. R. Rasmussen, and Amit Sahai. 2018. Threshold Cryptosystems from Threshold Fully Homomorphic Encryption. In *Advances in Cryptology – CRYPTO 2018*. Springer International Publishing, 565–596.
- [10] Jean-Philippe Bossuat, Christian Mouchet, Juan Troncoso-Pastoriza, and Jean-Pierre Hubaux. 2021. Efficient Bootstrapping for Approximate Homomorphic Encryption with Non-sparse Keys. In *Advances in Cryptology – EUROCRYPT 2021*. Springer International Publishing, 587–617.
- [11] Jean-Philippe Bossuat, Juan Troncoso-Pastoriza, and Jean-Pierre Hubaux. 2022. Bootstrapping for Approximate Homomorphic Encryption with Negligible Failure-Probability by Using Sparse-Secret Encapsulation. In *Applied Cryptography and Network Security*. Springer International Publishing, 521–541.
- [12] Christina Boura, Nicolas Gama, Mariya Georgieva, and Dimitar Jetchev. 2020. CHIMERA: Combining Ring-LWE-based Fully Homomorphic Encryption Schemes. *Journal of Mathematical Cryptology* 14, 1 (2020), 316–338. doi:10.1515/jmc-2019-0026
- [13] Zvika Brakerski. 2012. Fully Homomorphic Encryption without Modulus Switching from Classical GapSVP. In *Advances in Cryptology – CRYPTO 2012*. Springer, Berlin, Heidelberg, 868–886.
- [14] Hao Chen, Ilaria Chillotti, and Yongsoo Song. 2019. Improved Bootstrapping for Approximate Homomorphic Encryption. In *Advances in Cryptology – EUROCRYPT 2019*. Springer International Publishing, 34–54.
- [15] Hao Chen, Wei Dai, Miran Kim, and Yongsoo Song. 2019. Efficient Multi-Key Homomorphic Encryption with Packed Ciphertexts with Application to Oblivious Neural Network Inference. In *Proceedings of the 2019 ACM SIGSAC Conference on*

- Computer and Communications Security (CCS '19)*. Association for Computing Machinery, New York, NY, USA, 395–412. doi:10.1145/3319535.3363207
- [16] Jung Hee Cheon, Wonhee Cho, Jaehyung Kim, and Damien Stehlé. 2023. Homomorphic Multiple Precision Multiplication for CKKS and Reduced Modulus Consumption. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (Copenhagen, Denmark) (CCS '23)*. Association for Computing Machinery, New York, NY, USA, 696–710. doi:10.1145/3576915.3623086
- [17] Jung Hee Cheon, Hyeongmin Choe, Minsik Kang, Jaehyung Kim, Seonghak Kim, Johannes Mono, and Taeyeong Noh. 2024. Grafting: Decoupled Scale Factors and Modulus in RNS-CKKS. *Cryptology ePrint Archive*, Paper 2024/1014. <https://eprint.iacr.org/2024/1014>
- [18] Jung Hee Cheon, Hyeongmin Choe, Alain Passelègue, Damien Stehlé, and Elias Suvanto. 2024. Attacks Against the IND-CPAD Security of Exact FHE Schemes. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*. Association for Computing Machinery, New York, NY, USA, 2505–2519. doi:10.1145/3658644.3690341
- [19] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. 2018. Bootstrapping for Approximate Homomorphic Encryption. In *Advances in Cryptology – EUROCRYPT 2018*. Springer International Publishing, 360–384.
- [20] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. 2019. A Full RNS Variant of Approximate Homomorphic Encryption. In *Selected Areas in Cryptography – SAC 2018*. Springer International Publishing, 347–368.
- [21] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. 2017. Homomorphic Encryption for Arithmetic of Approximate Numbers. In *Advances in Cryptology – ASIACRYPT 2017*. Springer International Publishing, 409–437.
- [22] Jung Hee Cheon, Dongwoo Kim, and Duhyeon Kim. 2020. Efficient Homomorphic Comparison Methods with Optimal Complexity. In *Advances in Cryptology – ASIACRYPT 2020*. Springer International Publishing, 221–256.
- [23] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. 2016. Faster Fully Homomorphic Encryption: Bootstrapping in Less Than 0.1 Seconds. In *Advances in Cryptology – ASIACRYPT 2016*. Springer, Berlin, Heidelberg, 3–33.
- [24] Heewon Chung, Hyojun Kim, Young-Sik Kim, and Yongwoo Lee. 2024. Amortized Large Look-up Table Evaluation with Multivariate Polynomials for Homomorphic Encryption. *Cryptology ePrint Archive*, Paper 2024/274. <https://eprint.iacr.org/2024/274>
- [25] CryptoLab. 2022. HEAaN Library. Available at <https://heaan.it/>.
- [26] Nir Drucker, Guy Moshkovich, Tomer Pelleg, and Hayim Shaul. 2024. BLEACH: cleaning errors in discrete computations over CKKS. *Journal of Cryptology* 37, 1 (2024), 3.
- [27] Léo Ducas and Daniele Micciancio. 2015. FHEW: Bootstrapping Homomorphic Encryption in Less Than a Second. In *Advances in Cryptology – EUROCRYPT 2015*. Springer, Berlin, Heidelberg, 617–640.
- [28] J. Fan and F. Vercauteren. 2012. Somewhat Practical Fully Homomorphic Encryption. *Cryptology ePrint Archive*, Paper 2012/144. <https://eprint.iacr.org/2012/144>
- [29] Craig Gentry. 2009. Fully homomorphic encryption using ideal lattices. In *Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing (STOC '09)*. Association for Computing Machinery, New York, NY, USA, 169–178. doi:10.1145/1536414.1536440
- [30] Charanjit S. Jutla and Nathan Manohar. 2022. Sine Series Approximation of the Mod Function for Bootstrapping of Approximate HE. In *Advances in Cryptology – EUROCRYPT 2022*. Springer International Publishing, 491–520.
- [31] Andrey Kim, Maxim Deryabin, Jieun Eom, Rakyong Choi, Yongwoo Lee, Whan Ghang, and Donghoon Yoo. 2023. General bootstrapping approach for RLWE-based homomorphic encryption. *IEEE Trans. Comput.* 73, 1 (2023), 86–96.
- [32] Jaehyung Kim, Jinyeong Seo, and Yongsoo Song. 2024. Simpler and Faster BFV Bootstrapping for Arbitrary Plaintext Modulus from CKKS. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*. Association for Computing Machinery, New York, NY, USA, 2535–2546. doi:10.1145/3658644.3670302
- [33] Seonghak Kim, Minji Park, Jaehyung Kim, Taekyung Kim, and Chohong Min. 2022. EvalRound Algorithm in CKKS Bootstrapping. In *Advances in Cryptology – ASIACRYPT 2022*. Springer Nature Switzerland, Cham, 161–187.
- [34] Kamil Klucznik and Giacomo Santato. 2023. On Circuit Private, Multikey and Threshold Approximate Homomorphic Encryption. *Cryptology ePrint Archive*, Paper 2023/301. <https://eprint.iacr.org/2023/301>
- [35] Lattigo 2024. Lattigo v6. <https://github.com/tuneinsight/lattigo>. EPFL-LDS, Tune Insight SA.
- [36] Joon-Woo Lee, Eunsang Lee, Yongwoo Lee, Young-Sik Kim, and Jong-Seon No. 2021. High-Precision Bootstrapping of RNS-CKKS Homomorphic Encryption Using Optimal Minimax Polynomial Approximation and Inverse Sine Function. In *Advances in Cryptology – EUROCRYPT 2021*. Springer International Publishing, 618–647.
- [37] Yongwoo Lee, Joon-Woo Lee, Young-Sik Kim, Yongjune Kim, Jong-Seon No, and HyungChul Kang. 2022. High-Precision Bootstrapping for Approximate Homomorphic Encryption by Error Variance Minimization. In *Advances in Cryptology – EUROCRYPT 2022*. Springer International Publishing, 551–580.
- [38] Baiyu Li and Daniele Micciancio. 2021. On the Security of Homomorphic Encryption on Approximate Numbers. In *Advances in Cryptology – EUROCRYPT 2021*. Springer International Publishing, 648–677.
- [39] Baiyu Li, Daniele Micciancio, Mark Schultz-Wu, and Jessica Sorrell. 2022. Securing Approximate Homomorphic Encryption Using Differential Privacy. In *Advances in Cryptology – CRYPTO 2022*. Springer Nature Switzerland, Cham, 560–589.
- [40] Huixian Li, Wenyu Mo, Chun Shen, and Liaojuan Pang. 2025. EvalComp: Bootstrapping Based on Homomorphic Comparison Function for CKKS. *Trans. Info. For. Sec.* 20 (Jan. 2025), 1349–1361. doi:10.1109/TIFS.2024.3516553
- [41] Adriana López-Alt, Eran Tromer, and Vinod Vaikuntanathan. 2012. On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption. In *Proceedings of the Forty-Fourth Annual ACM Symposium on Theory of Computing (STOC '12)*. Association for Computing Machinery, New York, NY, USA, 1219–1234. doi:10.1145/2213977.2214086
- [42] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. 2010. On Ideal Lattices and Learning with Errors over Rings. In *Advances in Cryptology – EUROCRYPT 2010*. Springer, Berlin, Heidelberg, 1–23.
- [43] Christian Mouchet, Juan Troncoso-Pastoriza, Jean-Philippe Bossuat, and Jean-Pierre Hubaux. 2021. Multiparty Homomorphic Encryption from Ring-Learning-With-Errors. *Proceedings on Privacy Enhancing Technologies* (2021).
- [44] Pratyay Mukherjee and Daniel Wichs. 2016. Two Round Multiparty Computation via Multi-key FHE. In *Proceedings, Part II, of the 35th Annual International Conference on Advances in Cryptology – EUROCRYPT 2016 - Volume 9666*. Springer-Verlag, Berlin, Heidelberg, 735–763.
- [45] Nikola Samardžić and Daniel Sanchez. 2024. BitPacker: Enabling High Arithmetic Efficiency in Fully Homomorphic Encryption Accelerators. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 137–150. doi:10.1145/3620665.3640397
- [46] Damien Stehlé, Ron Steinfeld, Keisuke Tanaka, and Keita Xagawa. 2009. Efficient Public Key Encryption Based on Ideal Lattices. In *Advances in Cryptology – ASIACRYPT 2009*. Springer, Berlin, Heidelberg, 617–635.
- [47] Hyewon Sung, Sieun Seo, Taekyung Kim, and Chohong Min. 2025. EvalRound+ Bootstrapping and Its Rigorous Analysis for CKKS Scheme. *IEEE Access* 13 (2025), 140847–140866. doi:10.1109/ACCESS.2025.3595177

A Using Trits Rather Than Bits

In this appendix, we extend the bit-based integer cleaning to trit-based, then suggest a corresponding additional parameter set.

Cleaning trits. An extension of cleaning from bits to integers was considered in [26] by using the inclusion of bounded integers in the set of real numbers, and relying on binary decomposition for cleaning. Chung *et al* [24] proposed to rather rely on complex roots of unity to encode bounded integers: an integer $k \in [0, t - 1]$ is represented as $\exp(2i\pi k/t) \in \mathbb{C}$. Cleaning is then performed using the function $g_t : x \mapsto ((t+1)x - x^{t+1})/t$. It may be checked that g_t evaluates the identity function on the $\exp(2i\pi k/t)$'s and that these values are roots of g_t' . These properties ensure that g_t increases the accuracy of the distinguished roots of unity. An alternative roots of unity cleaning function f_t was proposed in [8]: $x, y \mapsto (y^{t-1} + 2(t-1)x - (t-1)x^2y)/t$ and evaluated in $(x, y) = (x, \bar{x})$. Evaluating it requires homomorphic conjugation, but consumes less multiplicative depth when t or $t+1$ is a power of 2.

LEMMA A.1. *For all $\beta \in \{\exp(-2i\pi/3), 1, \exp(2i\pi/3)\}$ and $\varepsilon, \varepsilon'$ with $|\varepsilon|, |\varepsilon'| \leq 1$, we have:*

$$|f_3(\beta + \varepsilon, \bar{\beta} + \varepsilon') - \beta| \leq 3 \max(|\varepsilon|, |\varepsilon'|)^2.$$

We note that the bound does not require ε' to be $\bar{\varepsilon}$.

PROOF. Consider $x, y \in \mathbb{C}$. We have:

$$\begin{aligned} 3 \cdot f_3(x + \varepsilon, y + \varepsilon') &= (y + \varepsilon')^2 + 4(x + \varepsilon) - 2(x + \varepsilon)^2(y + \varepsilon') \\ &= 3 \cdot f_3(x, y) + 4\varepsilon(1 - xy) + 2\varepsilon'(y - x^2) \\ &\quad - 2\varepsilon^2y - 4\varepsilon\varepsilon'x + \varepsilon'^2 - 2\varepsilon^2\varepsilon'. \end{aligned}$$

Now, for $x = \beta$ and $y = \bar{\beta}$, we observe that $f_3(x, y) = 3x$, $1 - xy = y - x^2 = 0$. The result then follows from the triangle inequality. $\square$

```

1:  $\bar{ct} := \text{conj}(ct)$ ;
2:  $\bar{ct}_1 := \bar{ct} \otimes \bar{ct} \in \mathcal{R}_Q^3$  ;
3:  $\bar{ct}_2 := \text{Relin}(\bar{ct}_1) \in \mathcal{R}_Q^2$  ;
4:  $ct_3 := \text{Relin}(ct \otimes \bar{ct}_2) \in \mathcal{R}_Q^2$  ;
5:  $\bar{ct}_3 := \text{conj}(ct_3) \in \mathcal{R}_Q^2$  ;
6:  $ct_4 := \Delta \cdot \bar{ct}_2 + 4 \cdot \Delta^2 \cdot ct - 2 \cdot \bar{ct}_3 \in \mathcal{R}_Q^2$  ;
7:  $ct_{\text{out}} := \text{Rescale}_\Delta(ct_4) \in \mathcal{R}_{Q/\Delta}^2$  .

```

Figure 4: The thrifty approach for evaluating f_3 .

Digit cleaning with trits. In Section 3.2, we explained how to perform digit cleaning with binary decomposition. Using a higher base β reduces the number of ciphertexts to be handled in parallel during vectorized cleaning, by a factor $\log_2 \beta$. However, increasing β may induce extra multiplicative depth. Note that representing integers as complex roots of unity has been suggested to be preferable over representing integers via the inclusion $\mathbb{Z} \subset \mathbb{C}$ (see [8, 24]). Table 3 compares the costs of different cleaning functions for β -th roots of unity for different β . Interestingly, third roots of unity can encode $\log_2 3 \approx 1.58$ bits of information and can be cleaned for the same cost as bits.

β	$\log_2(\beta)$	cleaning function	mult.	conj.	mult. depth
2	1.00	$\frac{3}{2}x - \frac{x^3}{2}$	2	0	2
3	1.58	$\frac{4}{3}x - \frac{x^4}{3}$	2	0	2
3	1.58	$\frac{1}{3}x^2 + \frac{4}{3}x - \frac{2}{3}x^2\bar{x}$	2	2	2
4	2.00	$\frac{5}{4}x - \frac{x^5}{4}$	3	0	3
4	2.00	$\frac{1}{4}x^3 + \frac{3}{2}x - \frac{3}{4}x^2\bar{x}$	3	2	2

Table 3: Different cleaning functions for β -th root of unities.

Reconstruction. Reconstructing bits into an integer is direct, using a power-of-two linear combination. Combining third roots of unity into the corresponding integer does not require multiplication, as we can extract the imaginary part with conjugation: mapping $\exp(-2i\pi/3)$, 1 and $\exp(2i\pi/3)$ to $-1/2$, 0 and $1/2$, respectively. For $\beta = 4$, we could not find a reconstruction algorithm that would not require a multiplicative level. Note that at this stage, we are at the maximal scaling factor, so a multiplicative level consumes a large amount of modulus.

Extractions. For the integer-to-roots conversion, we can use the same exponential polynomial of approximation range of $[-\pi, \pi]$, by scaling the messages in the input range $I \in [-(\beta^k/2 - 1), \beta^k/2 - 1]$, resulting in erroneous roots of unities $\exp(2i\pi I/\beta^k)$, where $\beta = 3$.

For the roots-to-trits conversion, we extend the approach of Bae et al. [8] to trits. For any k , we consider the Lagrange interpolation polynomials $P_{3,\ell}^{(k)}$'s from $\exp(2i\pi \sum_{0 \leq j < k} 3^j I_j)$ to I_ℓ for all $0 \leq \ell <$

k . They turn out to be sparse. For concreteness, we give their shape below for $k = 3$ (i.e., for I 's that can take $3^3 = 27$ values).

$$\begin{aligned}
P_{3,0} &= x^9, \\
P_{3,1} &= \alpha_3 x^3 + \alpha_{12} x^{12} + \alpha_{21} x^{21}, \\
P_{3,2} &= \alpha_2 x^2 + \alpha_5 x^5 + \alpha_8 x^8 + \alpha_{11} x^{11} + \alpha_{14} x^{14} \\
&\quad + \alpha_{17} x^{17} + \alpha_{20} x^{20} + \alpha_{23} x^{23} + \alpha_{26} x^{26},
\end{aligned}$$

for some α_i 's in $\mathbb{R}$.

Cleanings. For cleaning the extracted trits, we use $f_3(x, y) = \frac{1}{3}y^2 + \frac{4}{3}x - \frac{2}{3}x^2y$ (in $(x, y) = (x, \bar{x})$). We have seen in Section 4 a thrifty approach for cleaning bits. We now extend it to trits.

We “hide” the constant multiplication by $1/3$ of f_3 in the rescaling, so that we only compute the integer polynomial $y^2 + 4x - 2x^2y$. We then proceed as described in Figure 4, starting from a ciphertext $ct \in \mathcal{R}_Q^2$ decrypting to a (noisy binary) vector $\vec{z}$ with scaling factor Δ .

THEOREM A.2. *The thrifty approach given in Figure 4 is correct.*

PROOF. We first recall that the function $f(x) = (\bar{x}^2 + 4x - 2x^2\bar{x})/3$ can clean the errors of $\bar{x}$ and x at the same time: the error for $\bar{x}$ does not need to be the complex conjugate of the error for x (see Lemma A.1). Even if x and $\bar{x}$ have independent errors, the function f doubles the accuracy of $x \in \{\exp(-2i\pi/3), 1, \exp(2i\pi/3)\}$.

Now, assume that the input ct decrypts to $\vec{z}$. The ciphertext $\bar{ct}$ computed at Step 1 decrypts to $\vec{w}$ with scaling factor Δ , i.e., it satisfies $\text{DFT} \circ \text{Dec}(\bar{ct}) = \Delta \cdot \vec{w}$. Using the observation above, we view $\vec{w}$ as an error-free variable that is close to $\vec{z}$. Now:

$$\text{DFT} \circ \text{Dec}(\bar{ct}_2) = \Delta^2 \cdot \vec{w} \odot \vec{w} + \vec{e}_{\text{Relin}},$$

for some small relinearization error $\vec{e}_{\text{Relin}}$. Then, the computation of ct_3 and $\bar{ct}_3$ give

$$\text{DFT} \circ \text{Dec}(ct_3) = (\Delta \vec{z}) \odot (\Delta^2 \cdot \vec{w} \odot \vec{w} + \vec{e}_{\text{Relin}}) + \vec{e}'_{\text{Relin}},$$

$$\text{DFT} \circ \text{Dec}(\bar{ct}_3) = (\Delta \vec{z}) \odot (\Delta^2 \cdot \vec{w} \odot \vec{w} + \vec{e}_{\text{Relin}}) + \vec{e}'_{\text{Relin}} + \vec{e}_{\text{conj}},$$

for other small errors $\vec{e}'_{\text{Relin}}$ and $\vec{e}_{\text{conj}}$. We observe that the term of interest is scaled by Δ^3 and that the error terms are at most scaled by Δ . This is also the case for the summands $\Delta \cdot \bar{ct}_2$ and $\Delta^2 \cdot ct$ at Step 6. Lemma A.1 allows us to complete the proof. $\square$

Parameter sets. Finally, we provide an alternative parameter set using trit-based extractions, extracting from 27 periods, i.e., a range $[-13, 13]$, with 2^{-49} failure probability.

#periods (K)	Fail. Prob.	Extract. Method	Cleaning
3^3 (13)	2^{-49}	Int2Roots2Trits	h_1 on 3 trits

Table 4: Alternative parameter set using trits.

Note that this approach can be extended by defining the period as a product of powers of 2 and 3, thereby ensuring an appropriate failure probability.